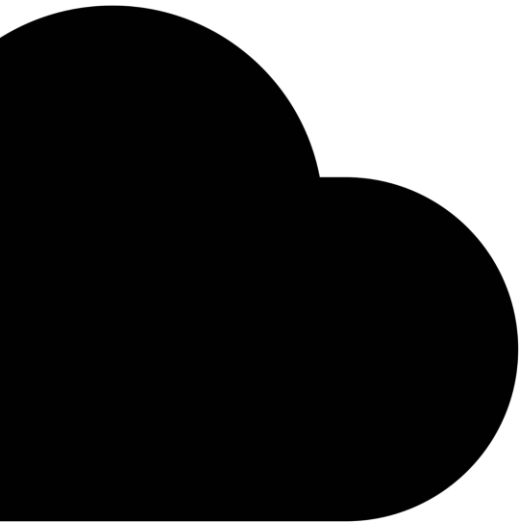


BESPIN GLOBAL

멀티클라우드 최종 프로젝트

BitComputer EMR

베스핀글로벌 | 팀 뿌듯



팀 구성 및 역할

홍 가 훈 SRE

M 010-4081-4408
E honggaheun@gmail.com
서울특별시 서초구 서초대로74길 33



Observability · DR · Cost Analysis

김 동 훈 DevOps Engineer

M 010-7400-5708
E kjbddo@gmail.com
서울특별시 서초구 서초대로74길 33



Kubernetes · CI/CD · GitOps

강 지 현 Cloud Infrastructure Engineer

M 010-2627-6570
E qounggcur@gmail.com
서울특별시 서초구 서초대로74길 33



Terraform · Iac

이 성 윤 Cloud Architect

M 010-5402-4200
E lucia25834@gmail.com
서울특별시 서초구 서초대로74길 33



Cloud Architect · System Design

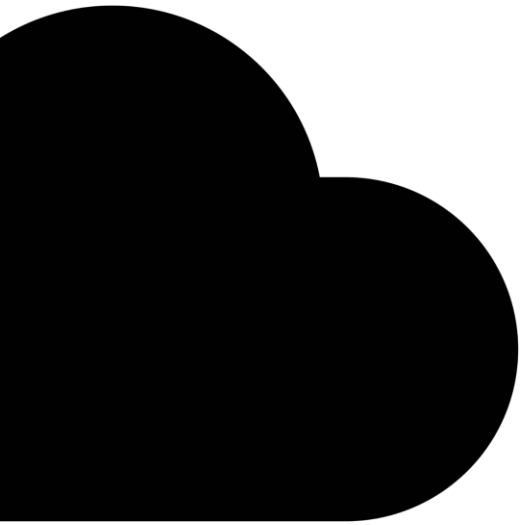
목차

I	EMR 서비스 · 인프라
II	Terraform
III	Kubernetes
IV	CI/CD · GitOps
V	Observability
VI	DR (GCP Pilot Light)
VII	OPSNOW
VIII	부록 1 EKS 버전 업그레이드 · 부록 2 상세 기술

BESPIN GLOBAL

EMR 서비스 · 인프라

이성윤



접수·진료·관리 서비스 EMR

진료 통합 서비스 / 영상·처방·진단서 **AI 서비스** / 환자 **개인정보**를 보호할 private 망이 필요합니다.



접수 · 진료 · 관리

환자 등록·대기 목록
상병·처방 코드 검색
진료 기록
사용자·부서·감사 로그



AI 처방 · 진단서

처방 그래프 DB
Bedrock Nova Pro 추천, 검증
에이전트가 큐에서 재검사, 진
단서 초안 생성

API : 직원 등록부터 진단서 저장까지

BitComputer EMR

bootstrap admin로그아웃

관리자 콘솔

감사 로그

부서 관리

직원 관리

부서 추가

새 부서명

내과

추가

부서 목록

ID	부서명	소속 인원	
1	UNASSIGNED	1	이름 수정

1

관리자 콘솔 · 부서와 직원 등록

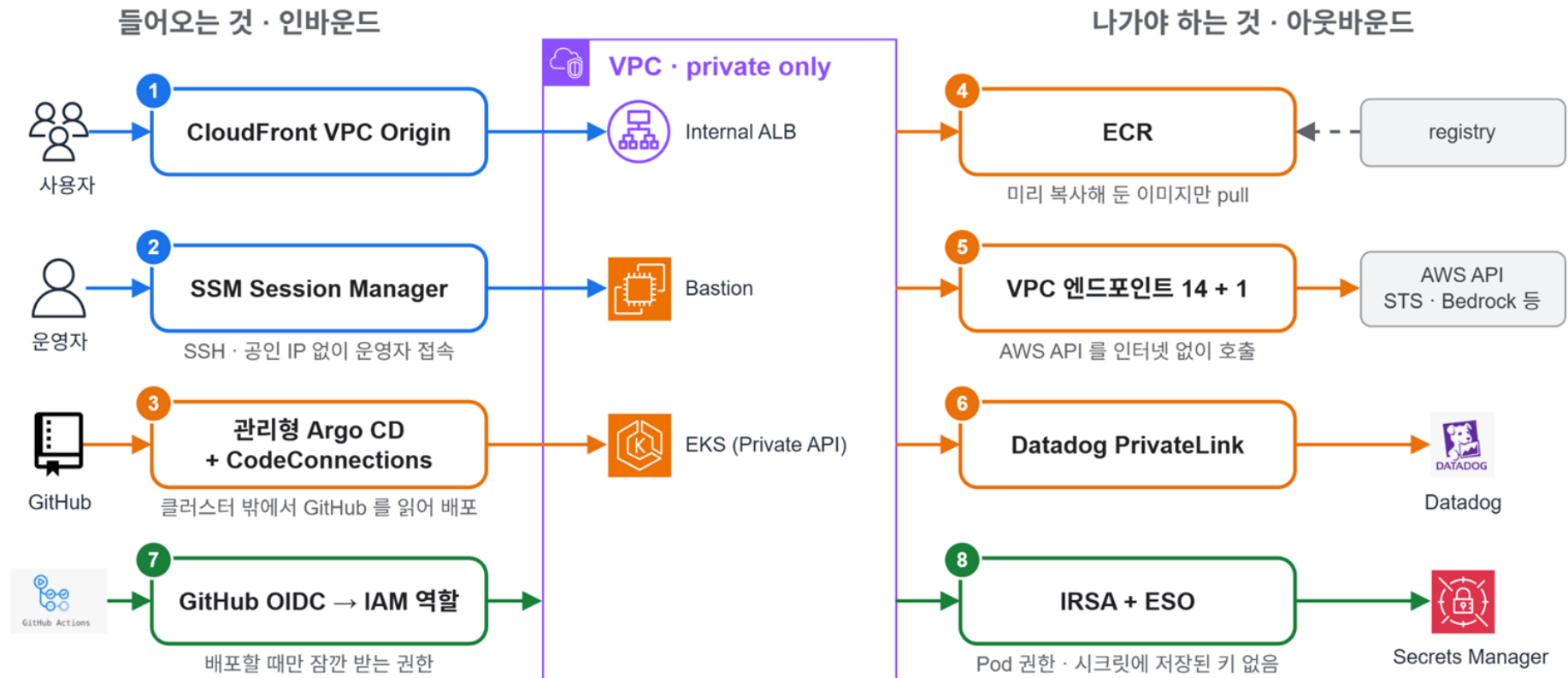
부서 '내과'를 만들고 의사 계정 doctor01 을 추가합니다

BitComputer EMR · www.bespin-global.cloud

1 / 8 ● ○ ○ ○ ○ ○ ○ ○ ○ ○ ○

prod 설계 원칙 : NAT 없는 인프라 아키텍처

“**public은 없다, NAT도 없다**” : 이 원칙이 모든 결정의 기준입니다.



NAT가 없다 - 비용과 보안의 트레이드오프

I EMR 서비스

AWS : 메인 인프라

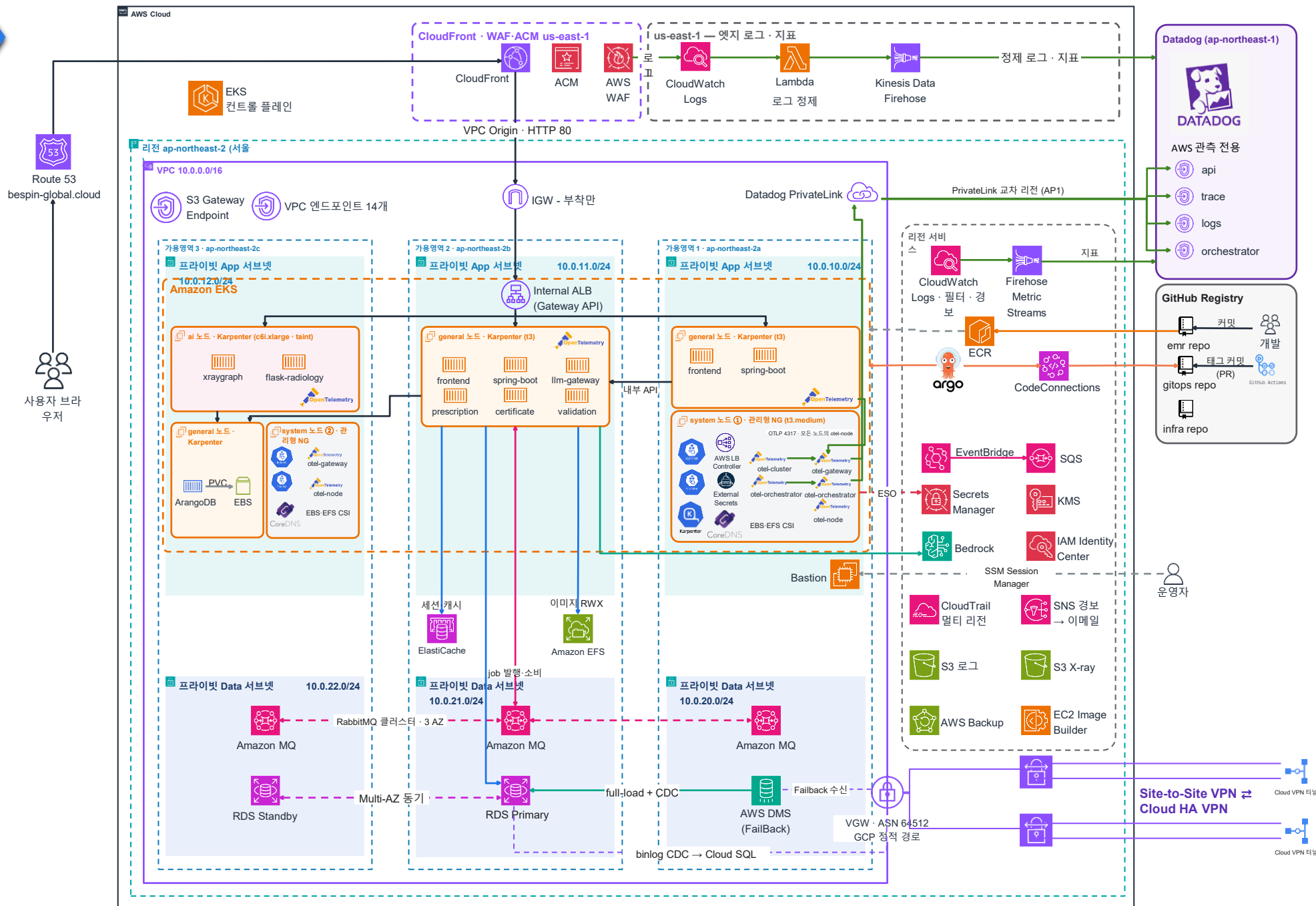
NAT 가 없는 폐쇄망 구조
: 제한된 아웃바운드

AWS 관리형 서비스
: VPC 엔드포인트로 연결

프라이빗 서브넷만 존재
: App Subnet 3 + Data Subnet 3

프라이빗 클러스터
: 비즈니스 로직 제공

DR 대비 GCP DMS
: GCP CloudSQL 이 주기적 플링



I EMR 서비스

AWS : 메인 인프라

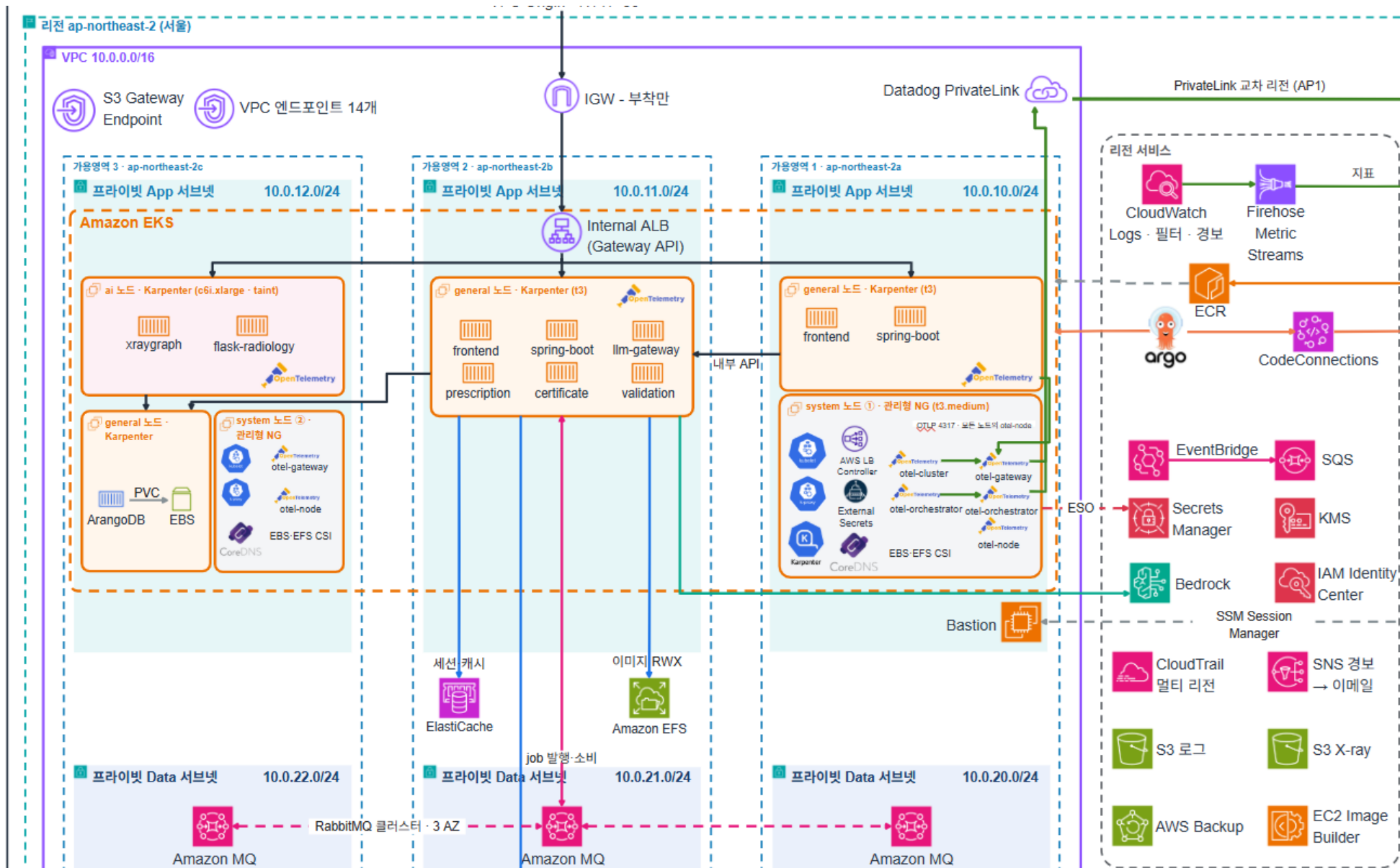
NAT 가 없는 폐쇄망 구조
: 제한된 아웃바운드

AWS 관리형 서비스
: VPC 엔드포인트로 연결

프라이빗 서브넷만 존재
: App Subnet 3 + Data Subnet 3

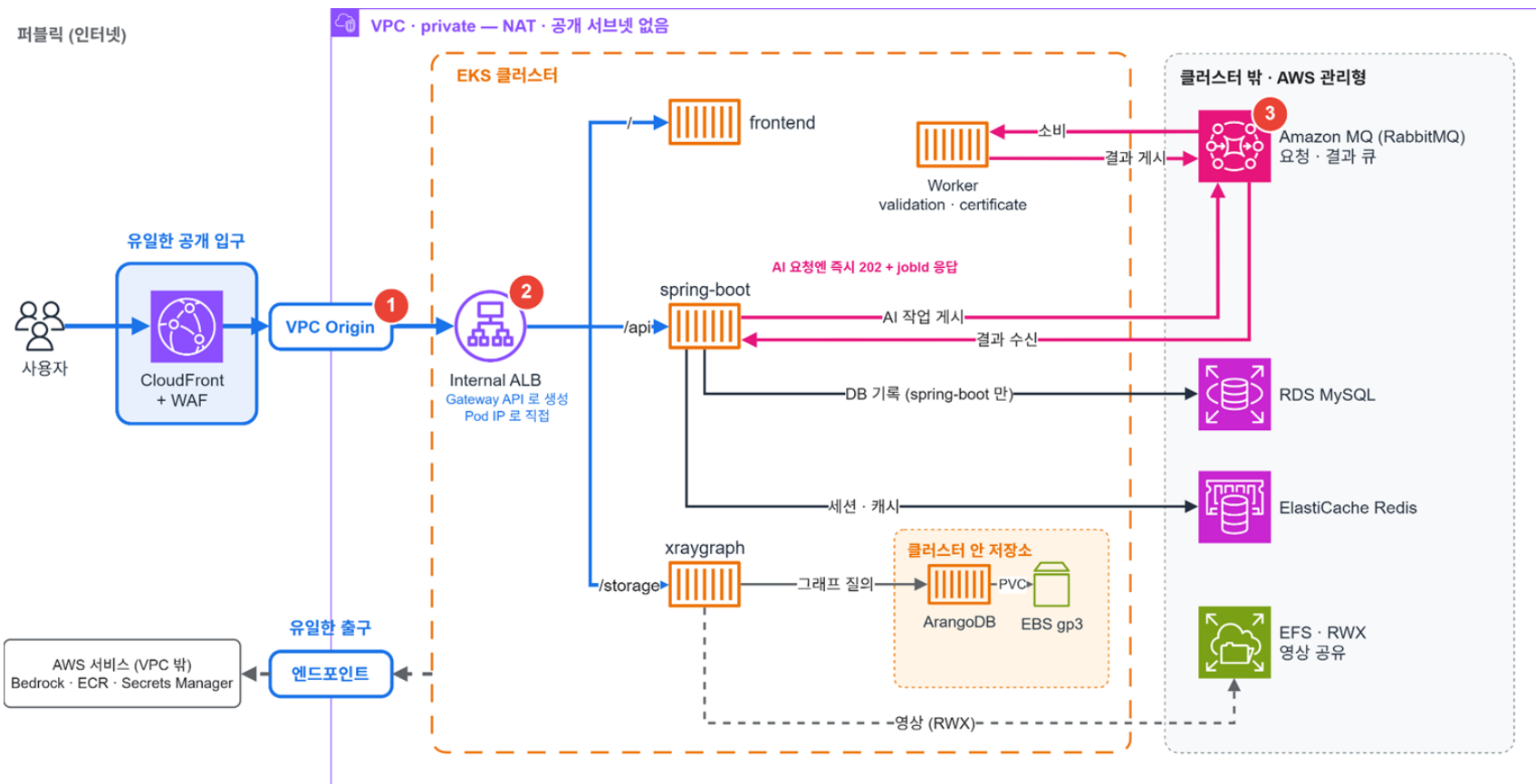
프라이빗 클러스터
: 비즈니스 로직 제공

DR 대비 GCP DMS
: GCP CloudSQL 이 주기적 폴링



서비스 플로우 : 진입점이 하나인 private 구조

외부의 진입점은 **CloudFront** 뿐입니다. 각 경로별 요청은 **Internal ALB**로 인해 3갈래로 이동합니다.



1 입구는 CloudFront 하나

VPC Origin을 거쳐 private 서브넷의 Internal ALB로 연결한다.

2 ALB가 경로로 3갈래

/ · /api · /storage 분류에 따라 해당 앱 Pod IP로 보낸다.

3 오래 걸리는 AI 작업은 큐로

MQ에 넘기고 다른 작업 즉시 응답, 결과는 큐로 돌아온다. (비동기처리)

I EMR 서비스

AWS : 메인 인프라

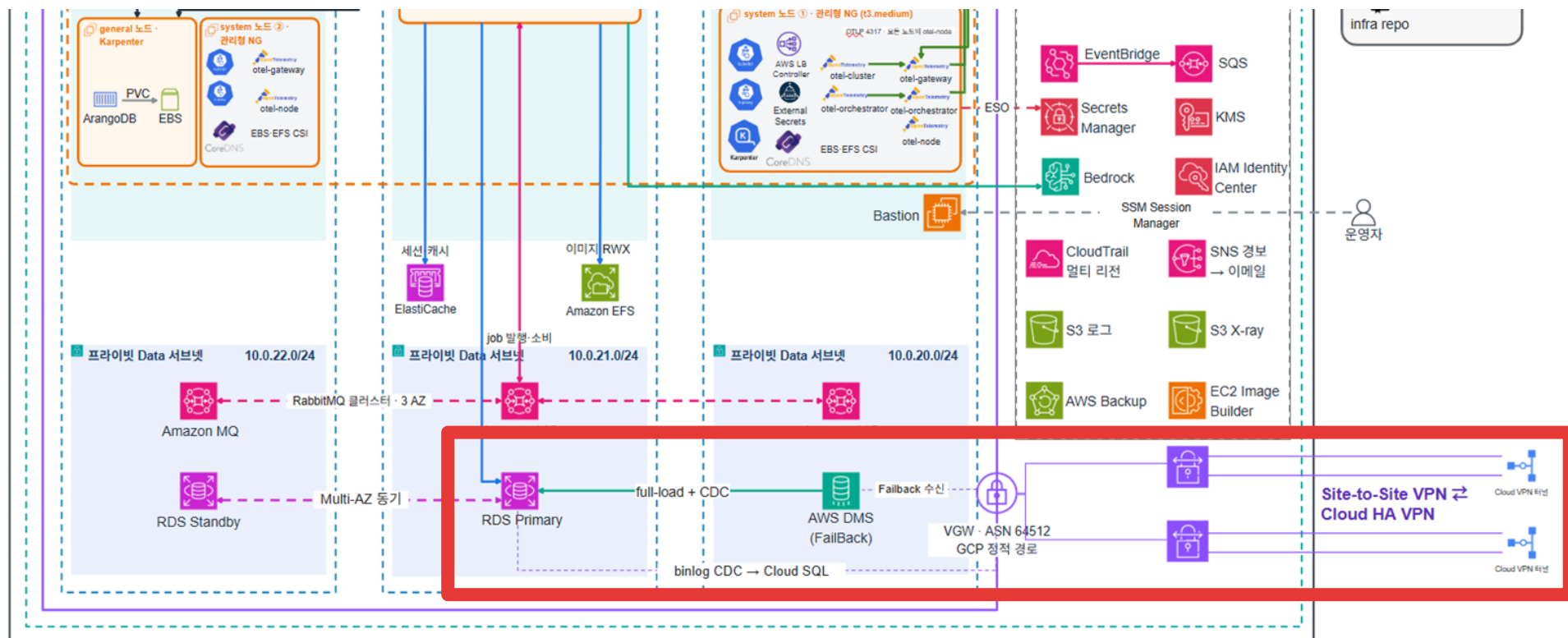
NAT 가 없는 폐쇄망 구조
: 제한된 아웃바운드

AWS 관리형 서비스
: VPC 엔드포인트로 연결

프라이빗 서브넷만 존재
: App Subnet 3 + Data Subnet 3

프라이빗 클러스터
: 비즈니스 로직 제공

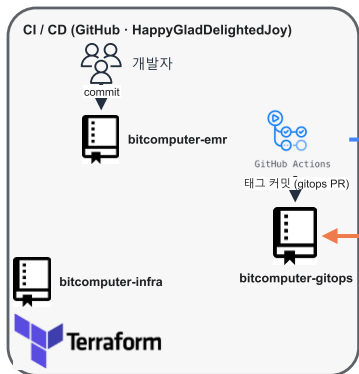
DR 대비 GCP DMS
: GCP CloudSQL 이 주기적 풀링



I EMR 서비스

GCP : DR 인프라

AI 서비스를 제외한 클러스터
: EMR서비스의 MVP 만 제공



운영자

Site-to-Site VPN ⇄ Cloud HA VPN



BESPIN GLOBAL

emr-platform — GCP DR (Pilot Light)

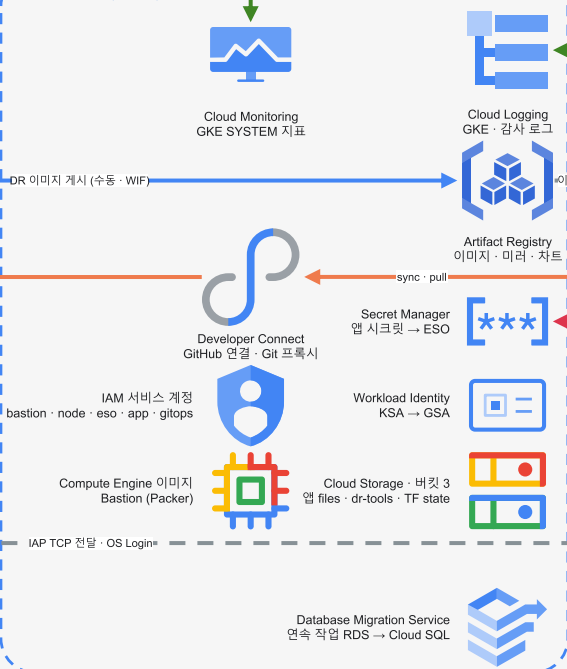


— 장애 시 수동 전환 · A 레코드 → GCP 고정 IP —

Google Cloud · 프로젝트 kdt4-4 (DR)



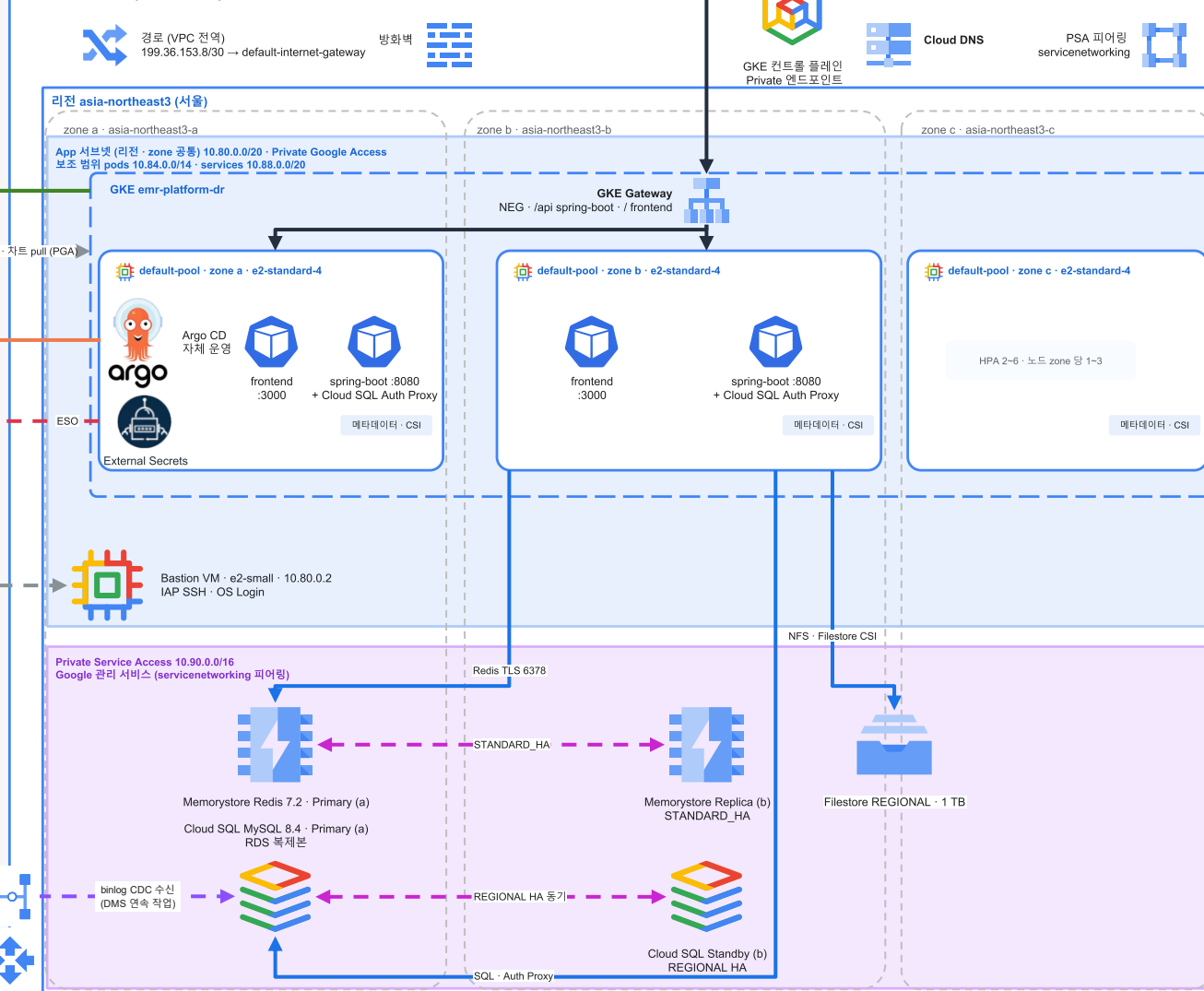
프로젝트 서비스 (VPC 밖)



Cloud VPN · HA VPN 게이트웨이
인터페이스 0 · 1

Cloud Router
터널마다 BGP 세션

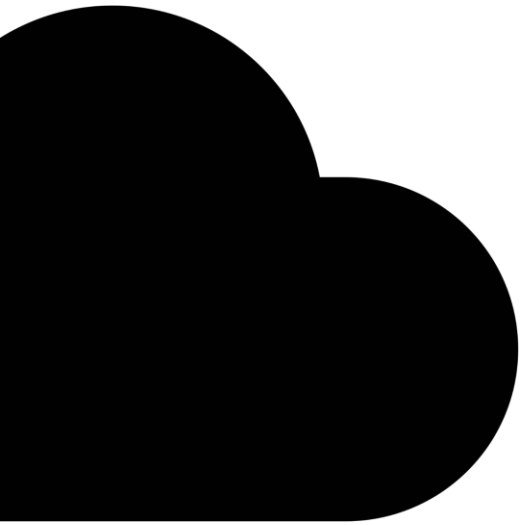
VPC 네트워크 emr-platform-dr-vpc · 글로벌 리소스 · 동적 라우팅 GLOBAL



BESPIN GLOBAL

Terraform

강지현



Terraform의 관리 범위

AWS 기반과 연결 권한은 Terraform이, EKS와 클러스터 안의 앱은 다른 도구가 맡습니다.



II Terraform

모듈은 역할별로, 실행은 의존 관계대로

bootstrap은 로컬 state로 먼저 실행하고, prod 모듈은 필요한 값을 주고받도록 연결합니다.

구분	모듈	만드는 것	먼저 필요한 것
시작	bootstrap	state 저장용 S3 backend 버킷 · ECR · GitHub Actions 인증/권한	별도 로컬 state
네트워크	network · endpoints	VPC · 사설 서브넷 · AWS 연결	bootstrap
보안	security	접근 규칙 · 암호화 키	network
데이터	data · DMS	DB · EFS · S3 · 백업 · 복구 연결	network · security
플랫폼	platform · karpenter · argocd	Bastion · CloudFront · EKS 연동 권한	기반 / EKS / ALB
애플리케이션	app · observability · VPN	앱 권한 · 로그 · 경보 · 선택적 VPN	필요한 기반 자원

※ 모듈은 기능과 변경 시점을 기준으로 나뉘었으며, 적용 순서는 자원 간 의존 관계로 결정됩니다.

state 파일은 두 개 : bootstrap & prod

bootstrap state는 로컬에, prod는 S3 backend에 state를 저장합니다.

state · 잠금 관리

bootstrap state : 로컬 파일

state 저장용 S3 backend 버킷을 만드는 기록

prod state : S3 파일 한 개

prod/main/terraform.tfstate
모든 모듈은 하나의 state를 공유

동시 작업 방지 : .tflock

use_lockfile로 작업 중 S3 잠금 파일 생성
정상 종료 시 삭제 · DynamoDB 미사용

보존형 철거와 재구축

철거 전 백업

RDS 스냅샷과 EBS · EFS 백업으로
복구 지점을 먼저 확보

state rm : 관리 목록에서만 제외

KMS 키 · Backup vault 등은 AWS에 남겨
나중에 데이터 복구에 사용

재구축 전 import

adopt_kept.py로 남긴 자원을 state에 연결
그다음 foundation apply로 중복 생성 방지

※ use_lockfile을 이용한 S3 잠금은 Terraform 1.10+에서 지원합니다. 이 프로젝트의 prod는 1.16+를 요구합니다.

II Terraform

구축 : 네 단계 plan, apply

EKS와 Internal ALB의 생성 시점에 맞춰, 단계마다 필요한 자원을 **누적**합니다.

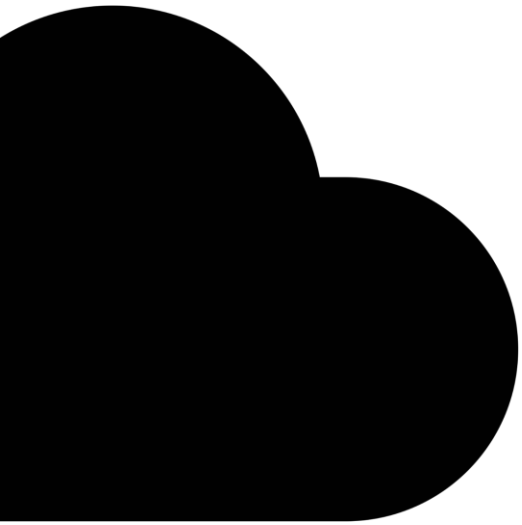


※ 숫자는 해당 단계 plan의 신규 생성 자원 수입입니다.

BESPIN GLOBAL

Kubernetes

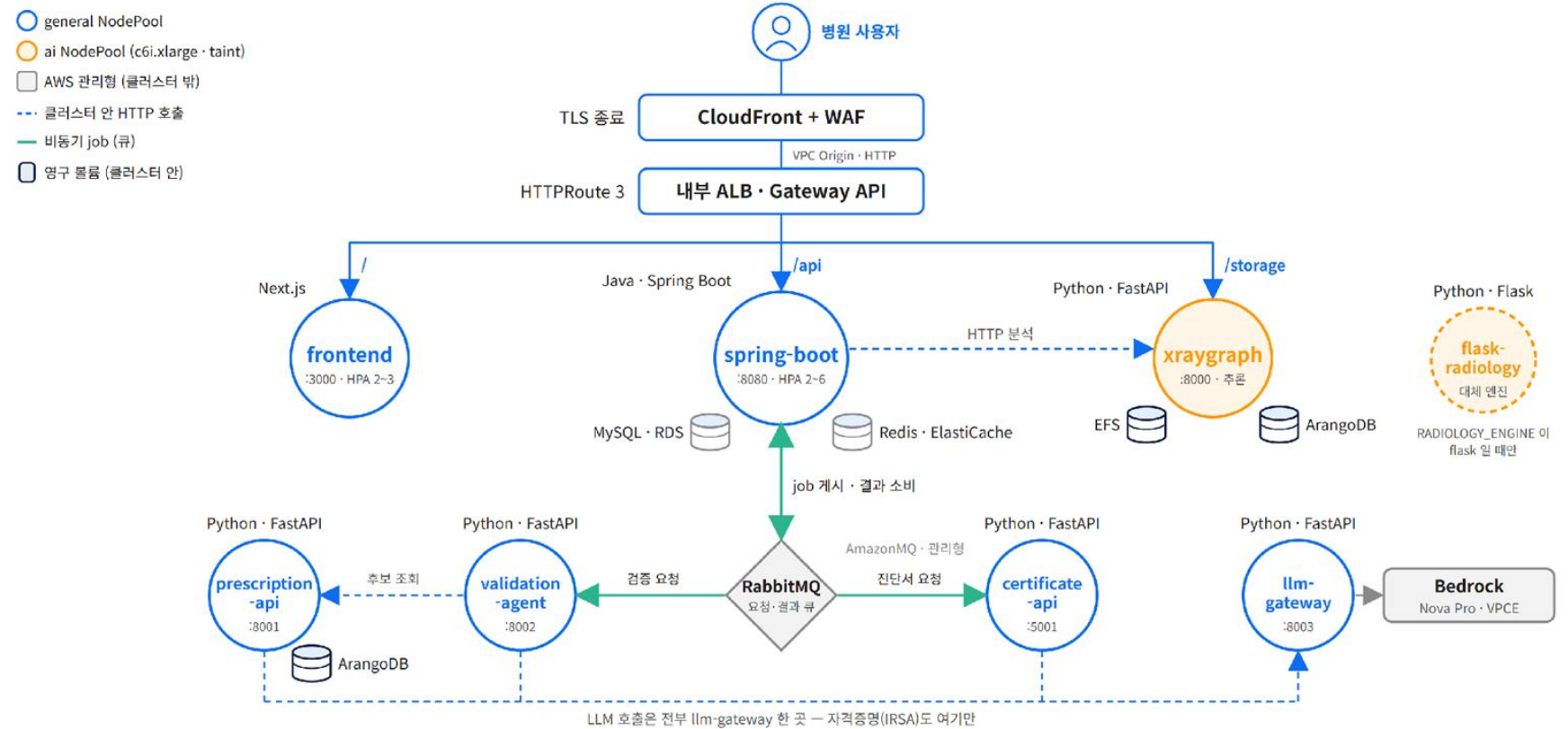
김동훈



클러스터 구조 : 비즈니스 로직

앱 8종

- 진입점 하나
CloudFront → 내부 ALB, 경로 3개
- 동기는 HTTP, AI 는 Queue
검증 · 진단서는 RabbitMQ로 넘김
- LLM 은 한 곳으로
자격증명은 llm-gateway 만
- 노드 풀로 격리
torch 추론 2종만 ai 노드



클러스터 안 · 영구 볼륨 (CSI)

ArangoDB — Deployment 1 · PVC → EBS gp3 (efs.csi)
영상 · X-ray 파일 — PVC → EFS RWX (efs.csi)
쓰는 앱: spring-boot · prescription · xraygraph · flask
노드가 바뀌어도 데이터는 볼륨에 남음

클러스터 밖 · AWS 관리형

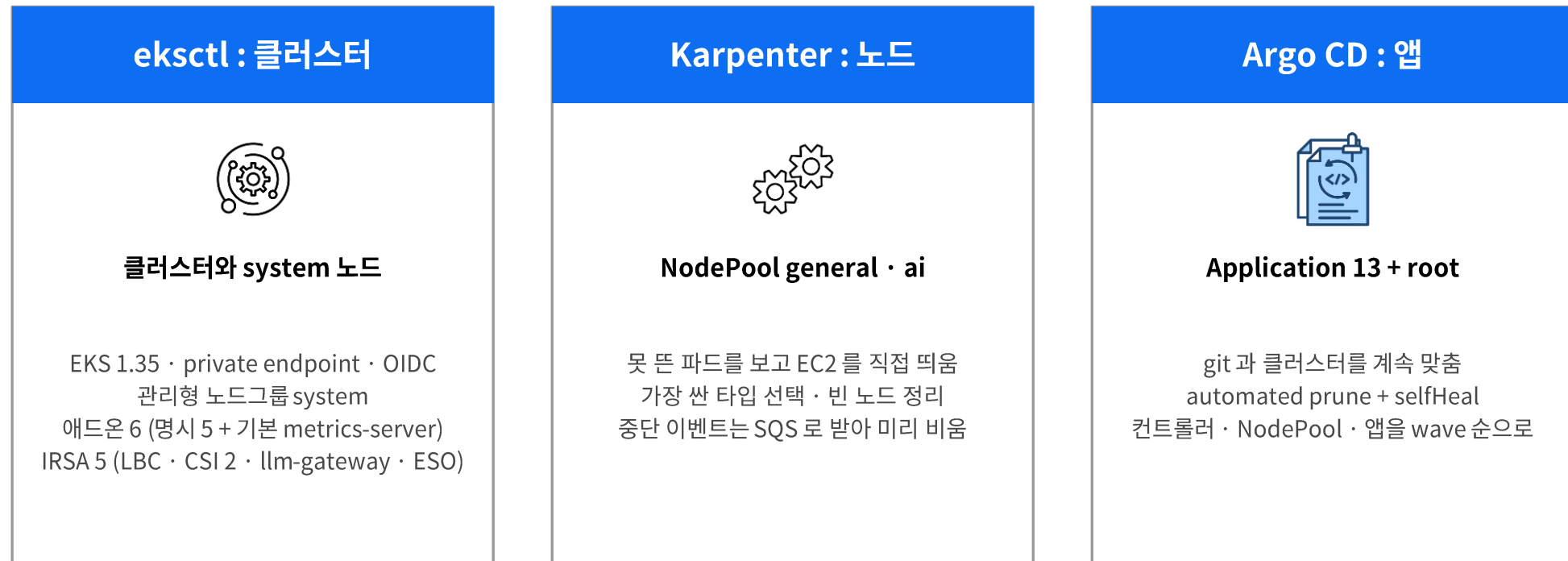
RDS MySQL (Multi-AZ) · ElastiCache Redis
AmazonMQ RabbitMQ (TLS 5671) · Bedrock
Secrets Manager → ESO 가 Secret 으로 동기화

system 노드 · 관리형 노드그룹 2대

Karpenter · AWS LBC · ESO
CoreDNS · metrics-server (EKS 애드온)
Argo CD 는 관리형 — 클러스터 밖 AWS 제어면

클러스터는 eksctl, 노드는 Karpenter, 앱은 Argo CD 가 만듭니다

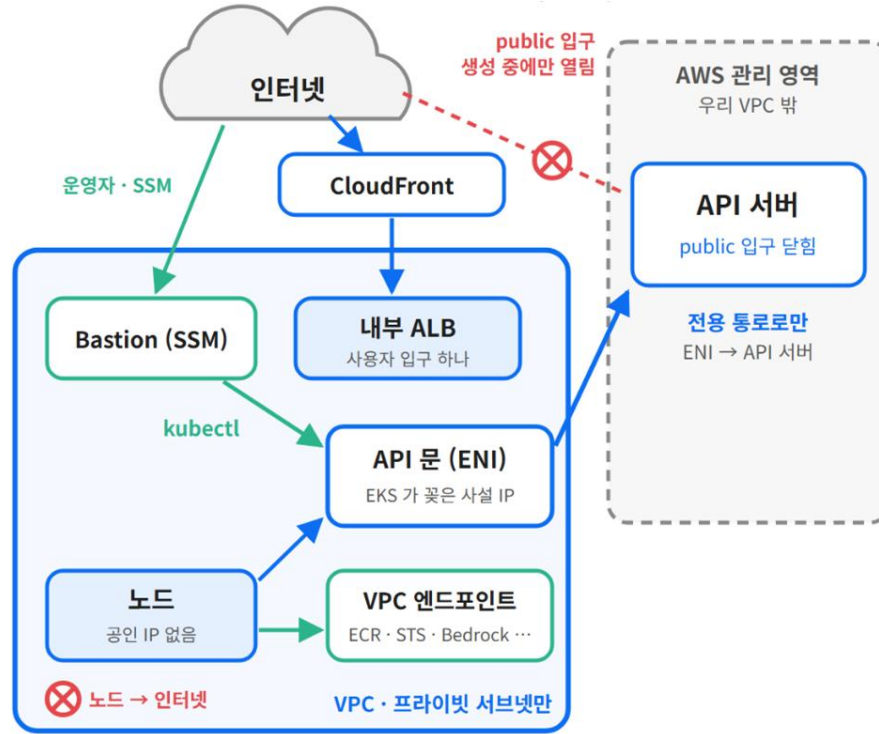
세 도구가 **만드는 대상이 겹치지 않게** 나뉘었습니다. 누가 무엇을 소유하는지가 곧 장애 때 볼 곳입니다.



“ 사람이 시작하는 것은 **eksctl** 두 번과 **bootstrap** 매니페스트 3개뿐. ”

프라이빗 클러스터

쿠버네티스 API 서버를 부르는 곳과 노드가 나가는 곳을 둘 다 VPC 안으로 닫았습니다.



문은 하나, 나머지는 전용 통로

API 서버는 VPC 밖 · 우리 VPC 엔 API 문(ENI)만 · 인터넷 라우트 0 · NAT 0

구분	퍼블릭 (EKS 기본)	프라이빗 (우리)
API 서버	인터넷에 공개	VPC 안에서만
kubectl	노트북 · CI 어디서나	Bastion(SSM) 한 곳
노드 서브넷	퍼블릭 · NAT 뒤	프라이빗 · 공인 IP 없음
노드 → 밖	NAT · IGW 로 인터넷	엔드포인트로 AWS 만
이미지	어디서나 pull	ECR 만 (외부는 미리)
대가	설정이 쉬움	엔드포인트 월 약 \$414

※ eksctl privateCluster 모드 · 서브넷 6개 전부 프라이빗 · 라우트 테이블에 0.0.0.0/0 없음 · NAT Gateway 0

NAT 가 없어서 따로 풀어야 했던 여섯 가지

인터넷이 필요한 일은 **클러스터 밖으로 옮기거나 미리** 해 두었습니다.

1 외부 이미지 · 차트

문제 노드가 Docker Hub · quay 에 못 감

해법 인터넷 되는 곳에서 ECR emr/mirror/* 로 미리 미러 (mirror.yml)

2 Argo CD 의 GitHub 읽기

문제 자체 호스팅이면 GitHub 에 못 닿음

해법 관리형 Argo CD → AWS 제어면에서 CodeConnections 로 읽음

3 Bastion 도구

문제 VPC 안에서 설치 파일을 못 받음

해법 kubectl · helm · eksctl 을 AMI 에 미리 구움 (Image Builder)

4 Karpenter 가격 · IAM API

문제 pricing · IAM 엔드포인트가 없음

해법 isolatedVPC: true · 인스턴스 프로파일은 Terraform 이 미리 생성

5 X-ray 모델 가중치

문제 자체 모델의 가중치를 인터넷에서 가져오지 못함

해법 이미지 빌드 때 가중치 선적재 → 첫 추론 3초

6 eksctl 의 클러스터 API 접근

문제 처음부터 닫으면 IRSA · 애드온 생성 실패

해법 privateCluster 모드 → 생성 중에만 public

Node Group & Node Pool 구성

관리형 노드그룹은 컨트롤러 전용. 워크로드는 Karpenter 가 띄우고 **torch 추론만 ai 로 격리**합니다.

구분	system (관리형 노드그룹)	general NodePool	ai NodePool
만드는 주체	eksctl	Karpenter (NodePool 은 Argo wave -1)	Karpenter
인스턴스	t3.medium	t3.small · medium · large 중 최저가	c6i.xlarge
규모	desired 2 · min 1 · max 2	하한 없음 · 상한 cpu 8 · mem 16Gi	하한 없음 · 상한 cpu 8 (최대 2대)
올라가는 것	Karpenter · LBC · ESO + kube-system	frontend · spring-boot · AI API 4 · ArangoDB	xraygraph · flask-radiology
격리	label workload=system	label workload=general	label + taint workload=ai:NoSchedule
교체	노드그룹 롤링 (콘솔 · eksctl)	drift · 통합 · 30일 만료	drift · 통합 · 30일 만료

※ system 을 2대로 둔 이유: t3.medium 은 파드 17개 한도라 컨트롤러가 한 대에 다 안 들어간다. 노드는 전부 on-demand · amd64

HPA : 파드 오토스케일링

기준은 requests 대비 CPU 70%. spring-boot 는 2~6 이고, Min이 1이면 재기동 순간 503 이라 최소가 2 입니다.

앱	Min	Max	requests → 70% 기준	PDB
spring-boot	2	6	250m → 평균 175m	1
frontend	2	3	100m → 평균 70m	1
llm-gateway	1	3	50m → 평균 35m	-
AI API 3종	1	2	50~100m	-
xraygraph · flask	1	2	500m · 100m (ai 노드)	-

예시 : 파드 2개 평균 210m(84%) → $[2 \times 84 \div 70] = 3$ 개

※ spring-boot max 6 은 운영 release 브랜치(gitops #82, 09/24). 차트 기본값은 3

1

metrics-server

EKS 관리형 애드온 · kubelet 의 파드 CPU 를 Metrics API 로

2

HPA 컨트롤러

15초마다 평균 사용률 계산 (기본값)

3

desired 계산

$[현재 \times 사용률 \div 70\%] \cdot min \sim max$ 사이로 자름

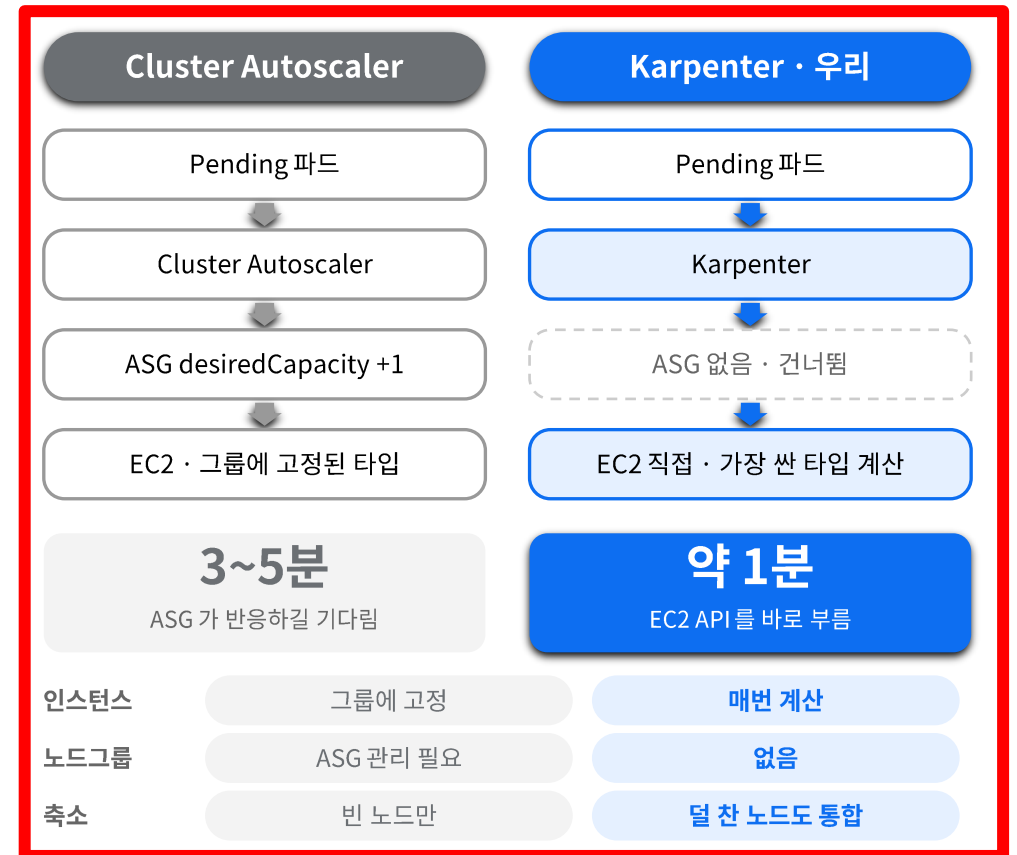
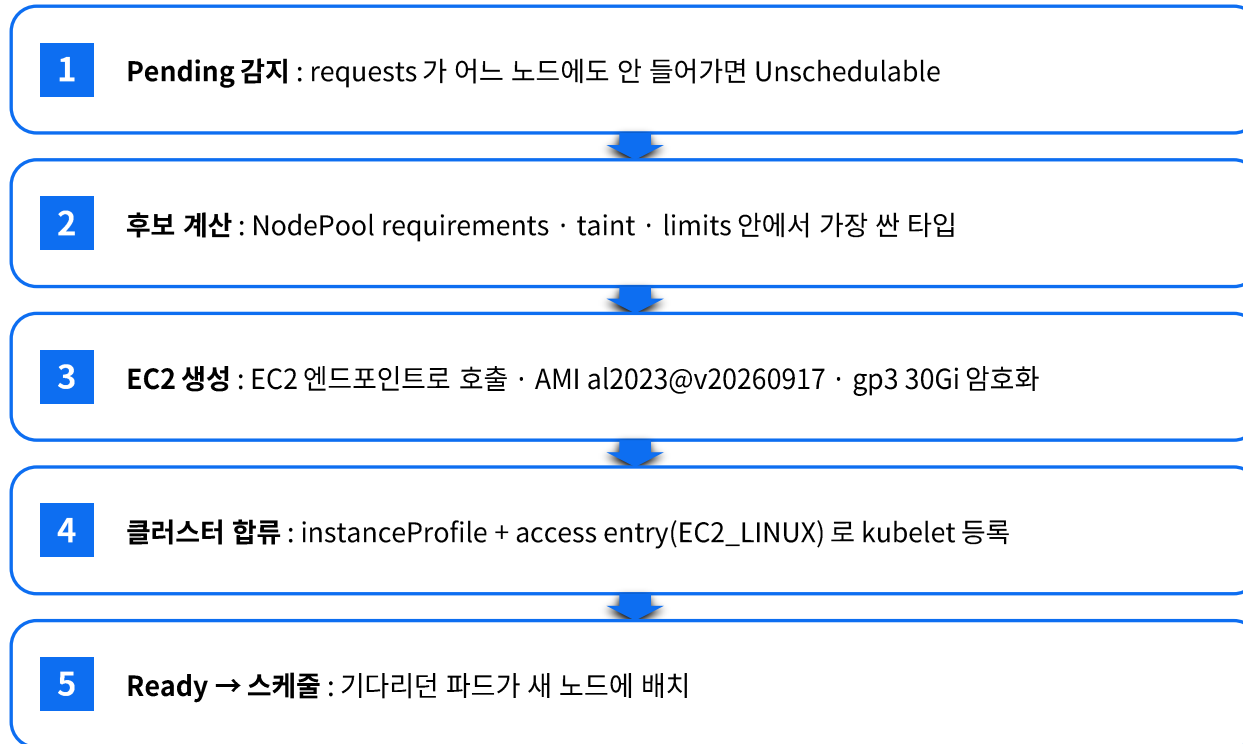
4

속도

늘릴 땐 바로, 줄일 땐 5분 지켜본 뒤 (behavior 미설정 = 기본값)

Karpenter : 노드 오토스케일링

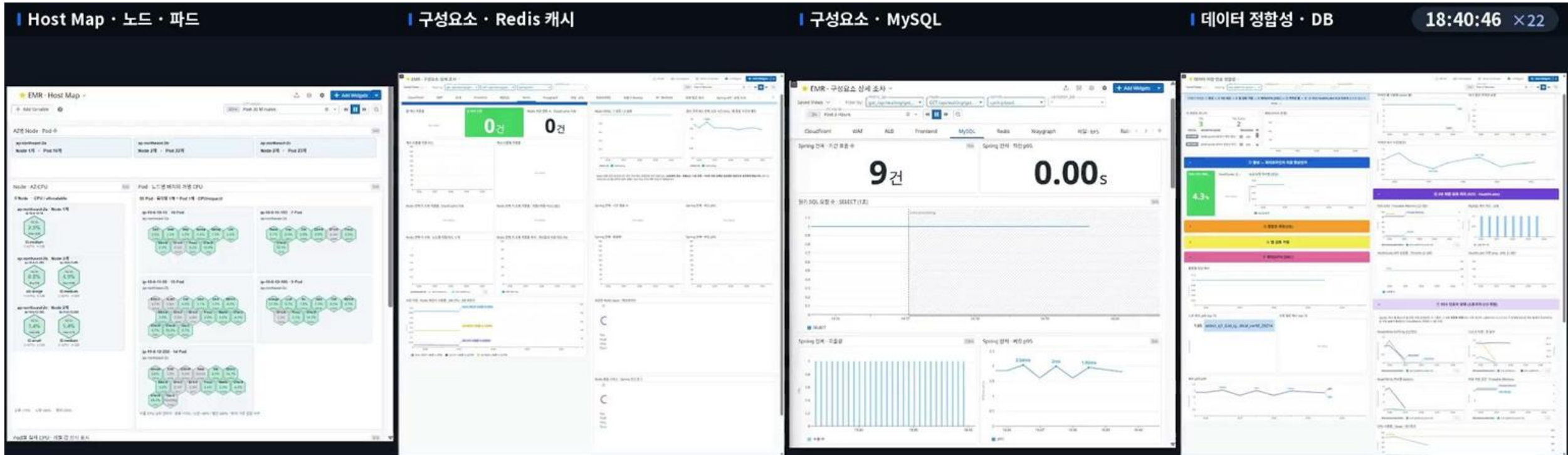
ASG 를 거치는 Cluster Autoscaler 와 달리 **EC2 를 직접 띄워** 노드그룹 없이 더 빠르게 늘리고 줄입니다.



※ role 대신 instanceProfile : IAM 엔드포인트가 없어 런타임 생성 불가 · 컨트롤러는 system 노드에 1개, 권한은 IRSA · 3~5분 · 약 1분은 일반값, 우리 실측은 파드 Ready 까지 약 2분(부하 테스트)

III Kubernetes

시연 : 부하가 파드와 노드로 번지는 모습



1

읽기 부하 · 대시보드 4개

같은 시각의 대시보드 4개 · 18:41 읽기 부하 시작 · 첫 요청이라 캐시 적중률 0%

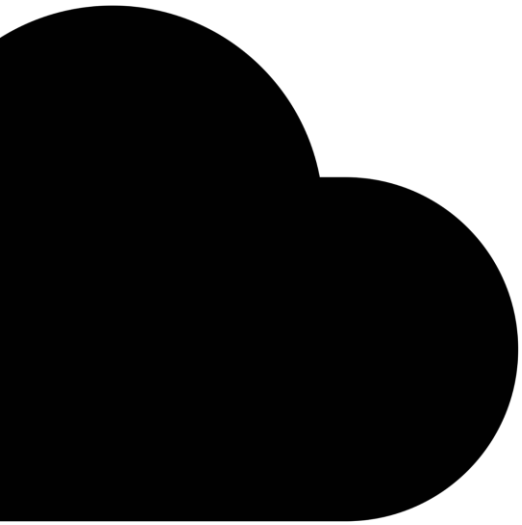
BitComputer EMR · www.bespin-global.cloud

1 / 8 ● ○ ○ ○ ○ ○ ○ ○ ○ ○ ○

BESPIN GLOBAL

CI/CD · GitOps

김동훈



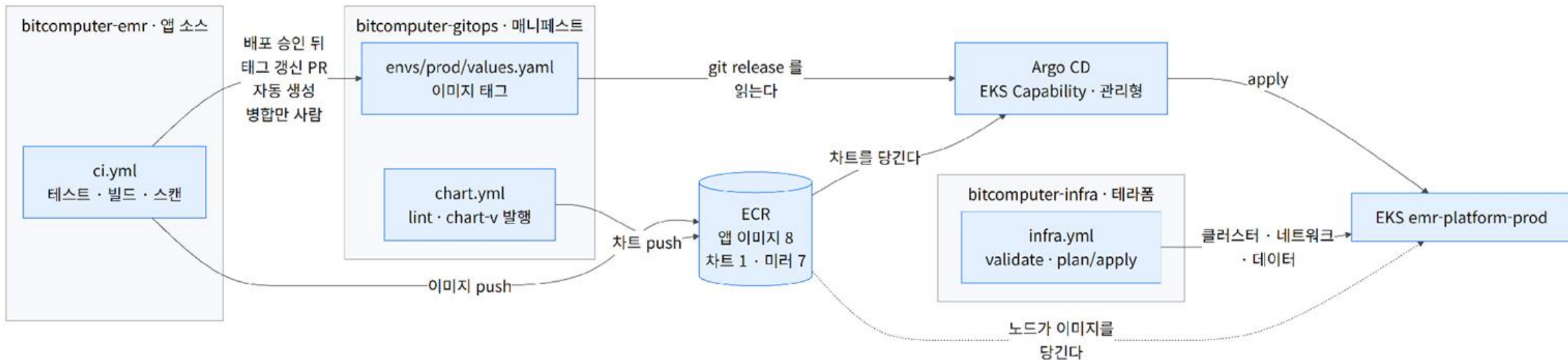
레포를 셋으로 나눈 기준은 코드가 아니라 크리덴셜입니다

레포끼리는 서로 부르지 않고 **ECR** 과 **git** 에 **놓인 산출물로만** 이어집니다. 이 경계가 곧 자격증명 경계입니다.



“ 앱은 이미지를, gitops 는 차트와 값을, infra 는 AWS 자원을 : **각자 자기 것만** 만듭니다 ”

CI/CD 한 장 : 커밋에서 파드까지 세 레포가 이어지는 길



1 태그는 sha-커밋7

불변 태그 하나만 쓰고 latest 는 만들지 않는다

2 병합 뒤에만 push

PR 은 빌드 · 스캔까지. 이미지는 release 병합 뒤 ECR 로

3 Argo CD 는 밖에서

관리형이라 클러스터 밖에서 git 과 ECR 을 당긴다

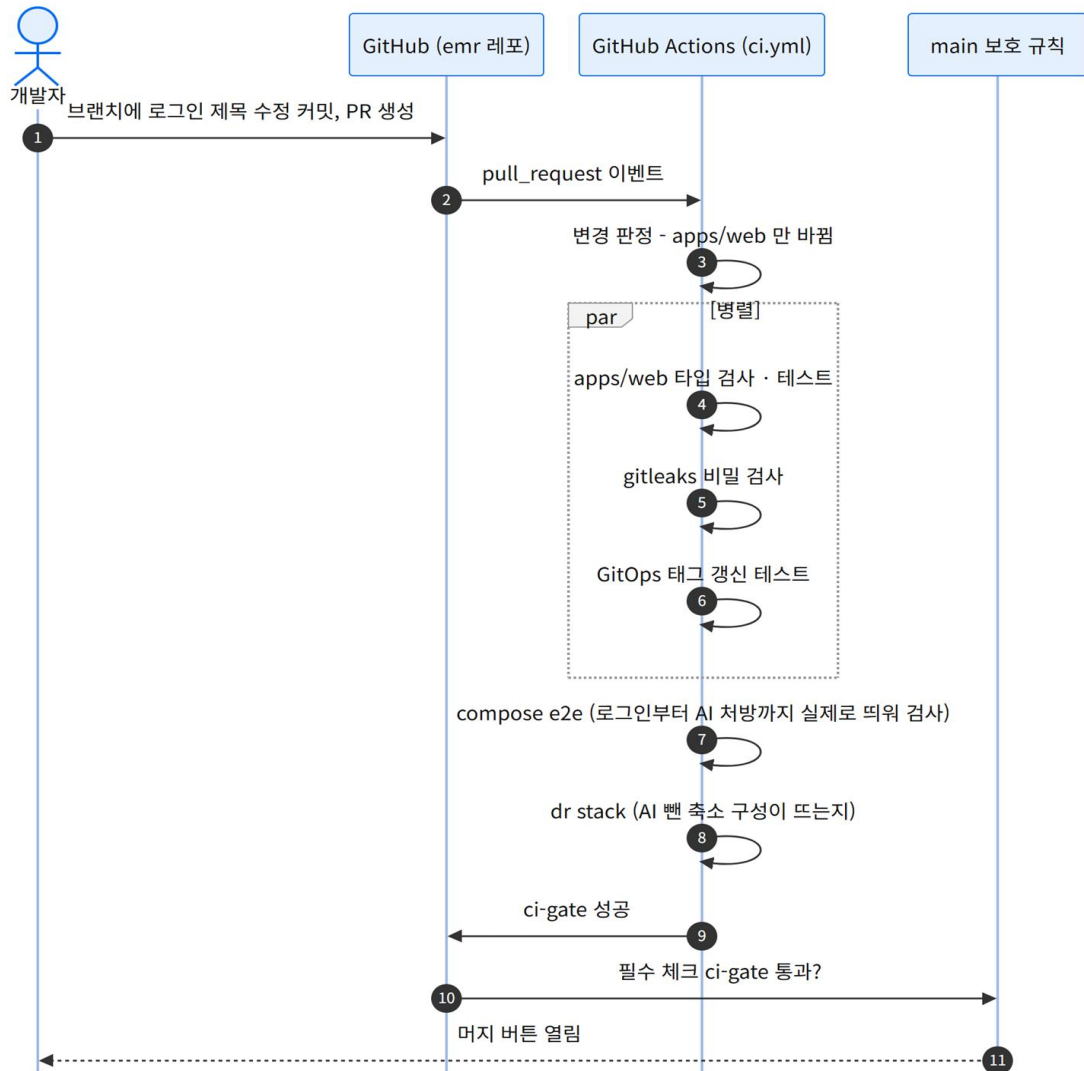
4 장수명 키 0

레포마다 OIDC 역할을 따로 쓴다

5 사람은 두 곳

태그 갱신 PR 병합 · 테라폼 apply 승인

emr ① CI : PR 발생 시, 검사를 다 통과해야 머지 버튼이 열립니다



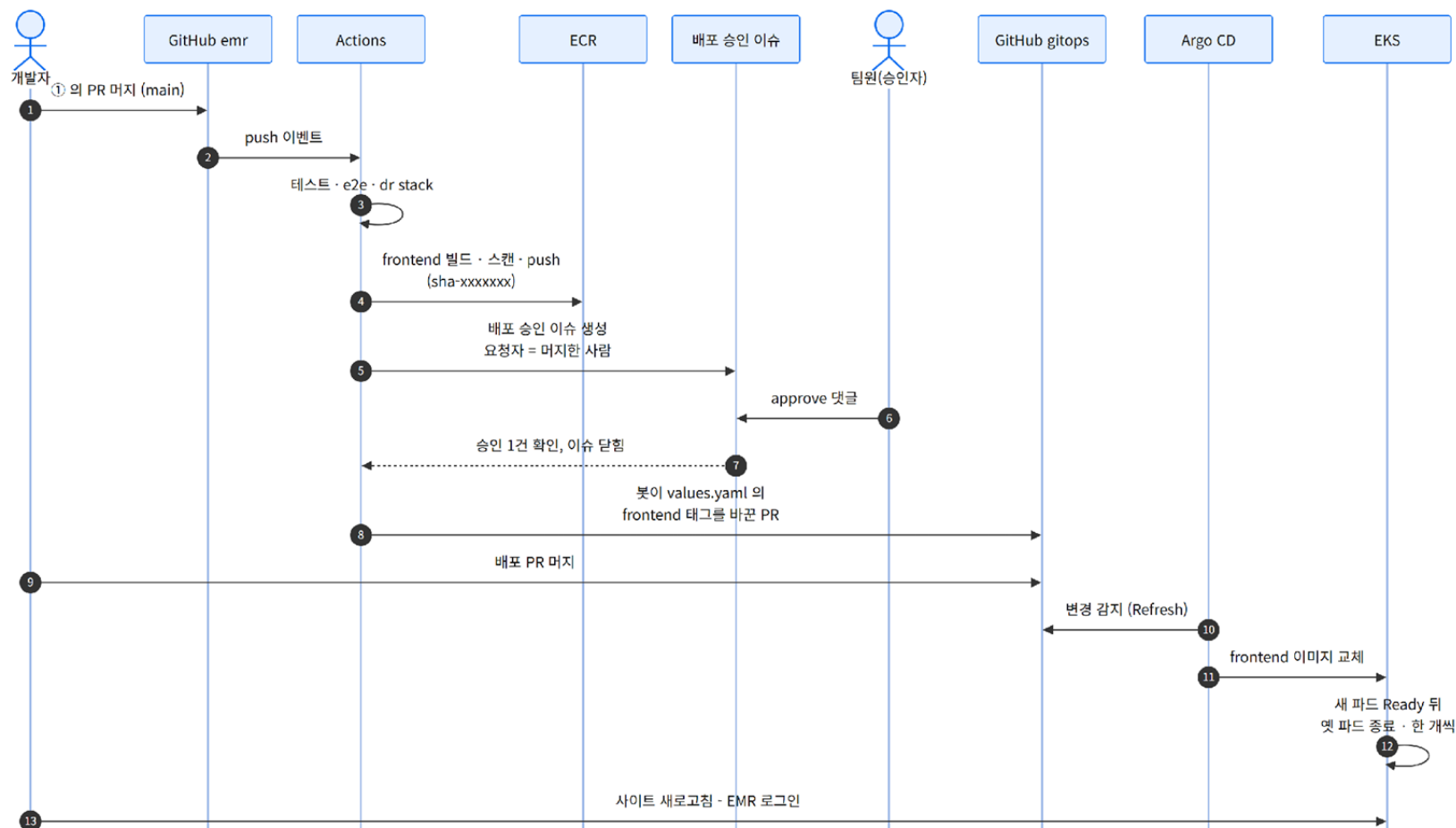
시연 ① 에서 보는 것

- ☐ 변경 판정이 먼저
- ☐ ci-gate 하나가 문지기
- ☐ PR 에션 굽지 않는다

변경 판정	apps/web	e2e · dr stack	ci-gate 까지
약 10초	약 2.5분	약 4~5분	약 8~10분

※ 2026-09-21 emr 실행 #25 실측

emr ② CD : 다른 사람의 승인과 사람의 머지를 거쳐 배포됩니다



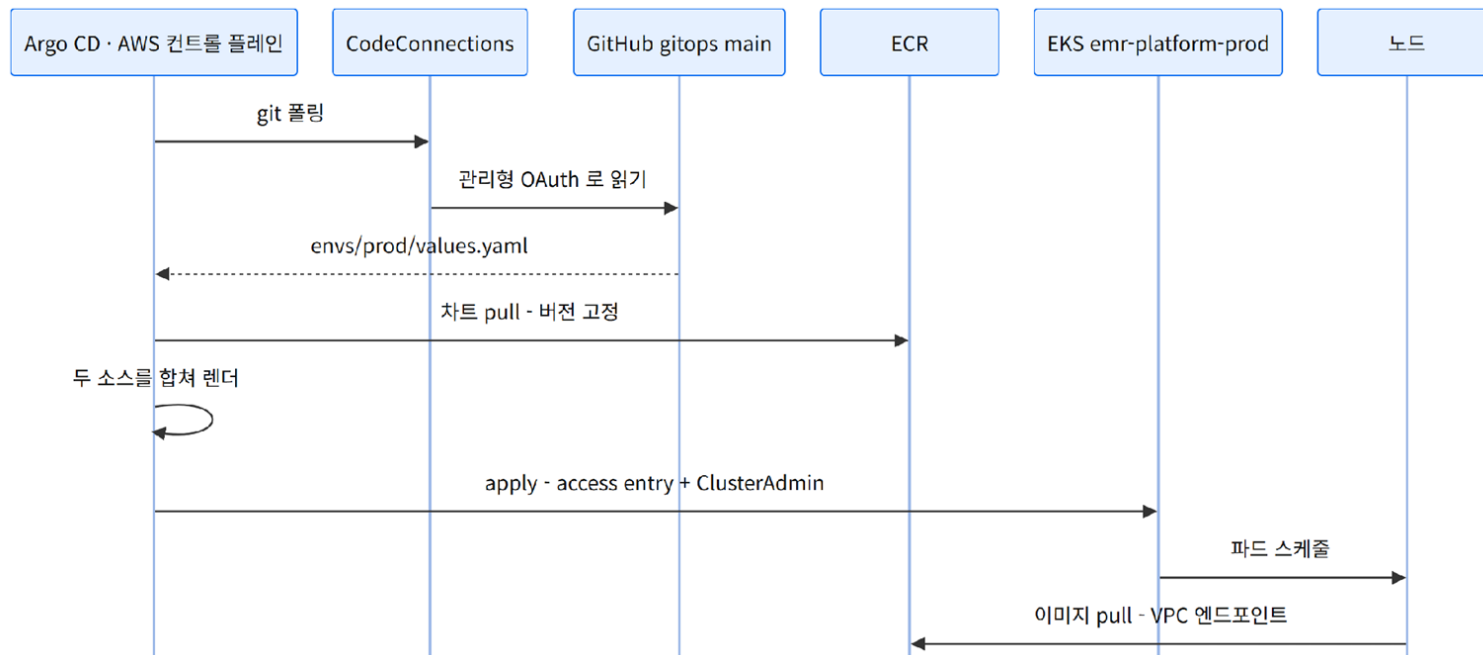
요점

- ❑ 바뀐 것만 굶는다
- ❑ 요청자 ≠ 승인자
- ❑ 봇은 제안까지

gitops ① Argo CD 가 git 과 ECR 을 당겨 클러스터에 적용합니다

클러스터 안에 GitHub 토큰도, Argo CD 파드도 없습니다.

CodeConnections가 git 을, **capability** 역할이 **ECR** 을 읽습니다.



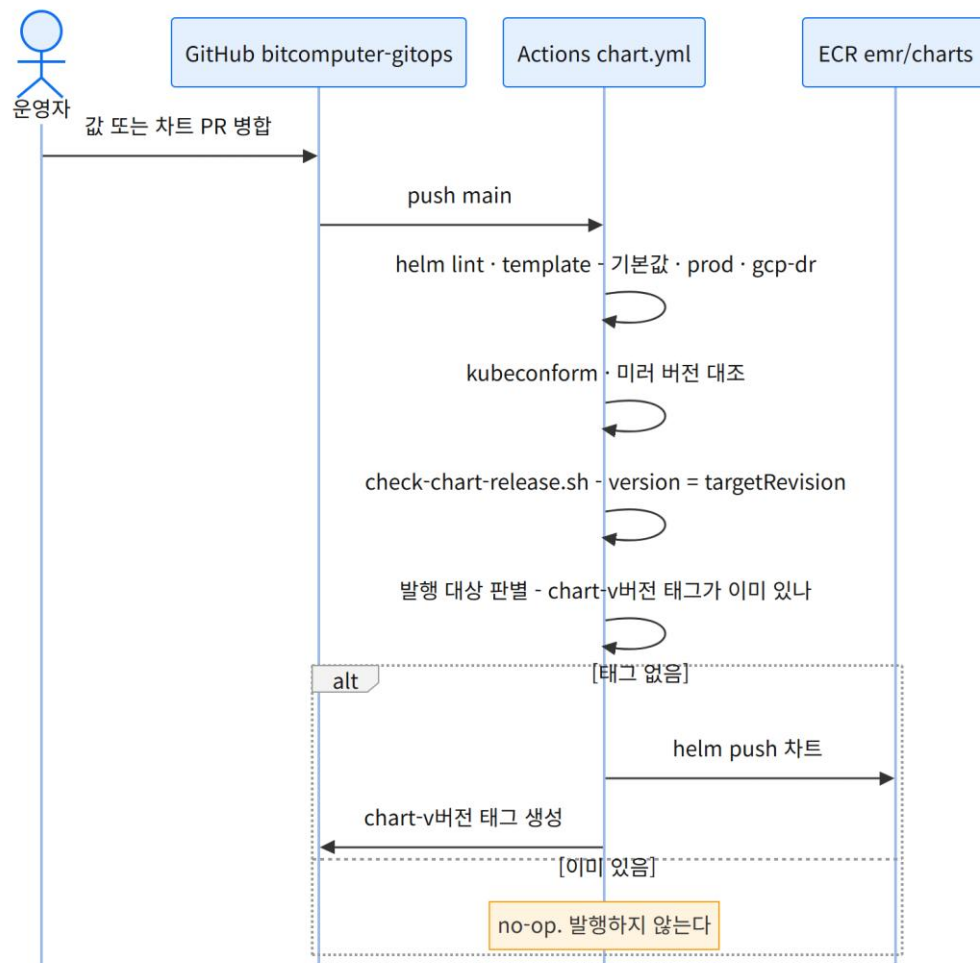
배포 방식

- ❑ 두 Source (chart + values)
- ❑ 풀링 지연 (4~6분)

※ 관리형이라 Argo CD 가 클러스터 밖(AWS 컨트롤 플레인)에서 돈다 : NAT 없는 VPC 에서도 GitHub 을 읽을 수 있는 이유

gitops ② 차트는 버전으로 고정하고, 태그는 CI가 찍습니다

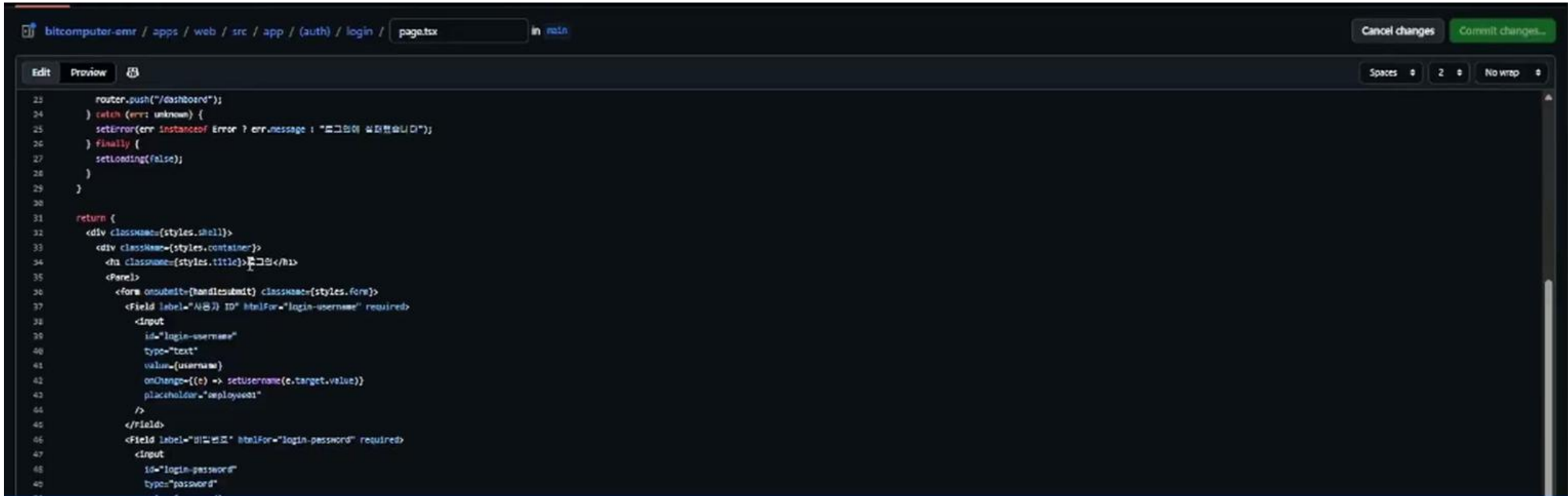
차트 **템플릿이 바뀔 때만** 버전이 바뀝니다. 이미지 태그만 바꿀 땐 차트를 다시 발행하지 않습니다.



발행 규칙

- ☐ 차트는 고정, 값만 바뀐다
- ☐ 버전 불일치는 PR에서 막는다
- ☐ 발행한 뒤에 태그
- ☐ 미러는 dispatch 로만

시연 : 코드 한 줄이 서비스에 닿기까지



The screenshot shows a code editor interface for a file named `page.tsx` located at `bitcomputer-emr / apps / web / src / app / (auth) / login /`. The editor has tabs for `Edit` and `Preview`, and buttons for `Cancel changes` and `Commit changes...`. The code is written in TypeScript and uses React with a form for login. The code includes a router push for the dashboard, error handling, and a form with fields for username and password. The code is as follows:

```
23 router.push("/dashboard");
24 } catch (err: unknown) {
25   setError(err instanceof Error ? err.message : "로그인에 실패했습니다.");
26 } finally {
27   setloading(false);
28 }
29 }
30
31 return (
32   <div className={styles.shell}>
33     <div className={styles.container}>
34       <h1 className={styles.title}>로그인</h1>
35       <Panel>
36         <form onSubmit={handleSubmit} className={styles.form}>
37           <Field label="사용자 ID" htmlFor="login-username" required>
38             <input
39               id="login-username"
40               type="text"
41               value={username}
42               onChange={(e) => setUsername(e.target.value)}
43               placeholder="employees"
44             />
45           </Field>
46           <Field label="비밀번호" htmlFor="login-password" required>
47             <input
48               id="login-password"
49               type="password"
50             />
51           </Field>
52         </form>
53       </Panel>
54     </div>
55   </div>
56 )
```

1

코드 수정 · 커밋

GitHub 에서 로그인 화면 제목 한 줄을 고쳐 새 브랜치로 커밋합니다

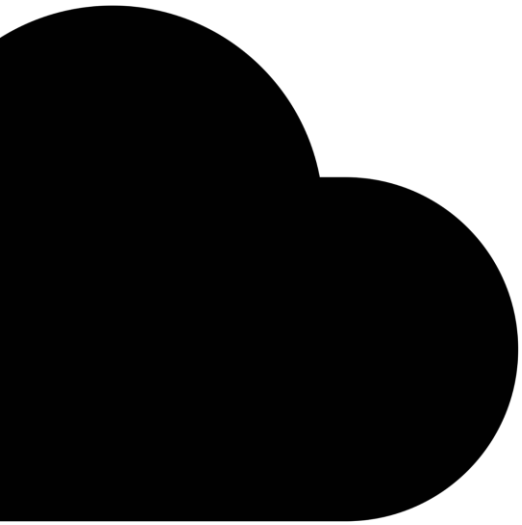
BitComputer EMR · www.bespin-global.cloud

1 / 9 ● ○ ○ ○ ○ ○ ○ ○ ○ ○ ○

BESPIN GLOBAL

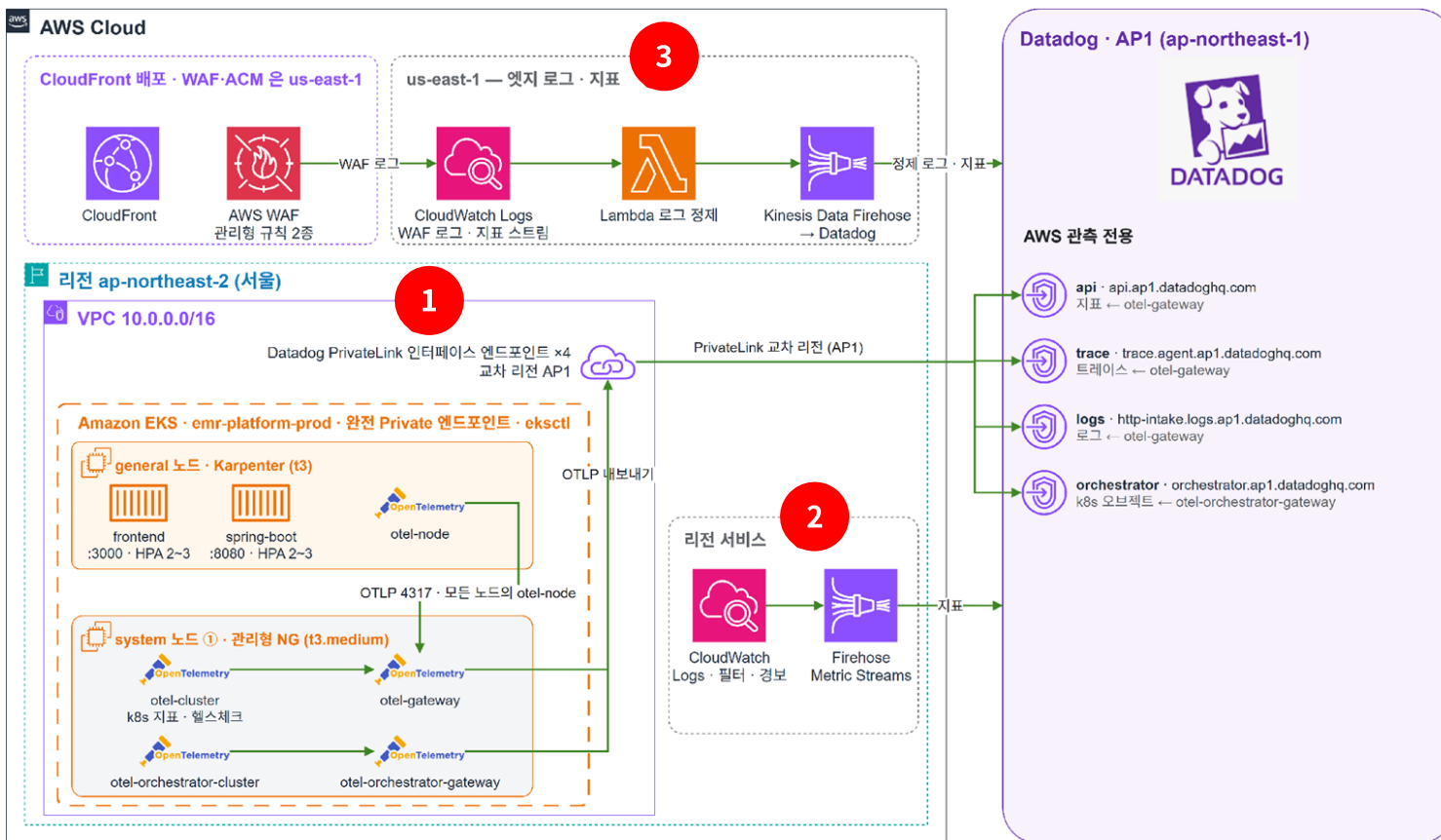
Observability

홍가흔



NAT 없이도 관측 데이터는 세 갈래로 나갑니다

EKS 안 수집기는 **PrivateLink**, AWS 관리형 지표·로그는 Firehose : 인터넷 egress X



1

EKS 내부

앱 계측(OTel) → Collector 3계층 → PrivateLink 4개

2

AWS 관리형 지표

RDS · ALB · CloudFront → Metric Streams → Firehose → Datadog

3

AWS 로그

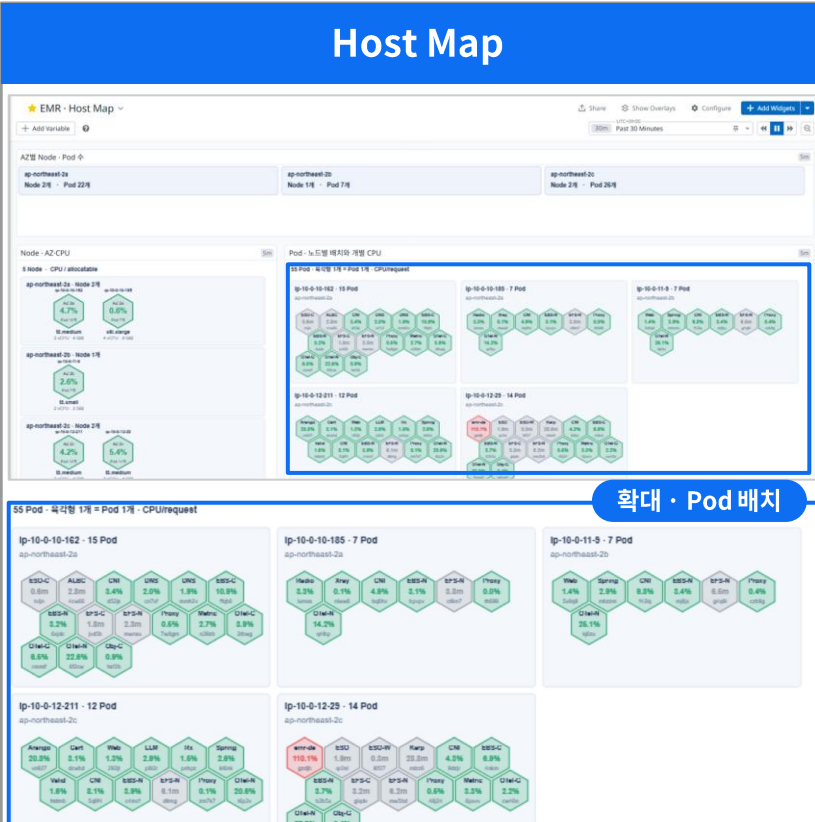
CloudFront · WAF 로그 → Lambda 정제 → Firehose

V Observability

Observability 대시보드

Host Map은 노드·Pod 확장 **핵심 업무 흐름**은 API 오류·지연 **구성요소 상세 조사**는 요청이 지나가는 구간별 지표

Host Map



- AZ별 노드·Pod 수, 노드 별집(CPU·EC2 유형)
- 부하 때 Pod·노드가 늘어나는 모습

핵심 업무 흐름



확대 · 요청 경로



- API별 요청 수·오류율·p95, 느린 API top 10
- 요청 표본에서 Trace·로그로 바로 이동

구성요소 상세 조사



- CloudFront→WAF→ALB→앱→DB 14그룹
- 구간별 네이티브 지표와 요청 증거

알림 : 22개

모니터 => SNS 토픽 => 메일

무엇을 알리나

묶음	개수	알려 주는 것	여는 대시보드
사용자 진입·API	4	사용자가 오류·지연을 겪는 중	핵심 업무 흐름
클러스터	4	파드·노드가 못 뜨거나 재시작 반복	Kubernetes · 구성요소 상세
데이터	5	DB 지연·실패, 저장 데이터 어긋남	데이터 저장·전송 적합성
비동기 AI	7	AI 작업이 쌓이거나 실패·중복	비동기 처방 검증·진단서
보안 (표시 전용)	2	권한 거부가 평소보다 잦음	보안 이벤트

어떻게 설정했나

[EMR prod] 핵심 API 5xx 비율

9/21 15:00 에 실제로 온 알림 메일

- ① 무엇이 핵심 API 5xx 비율 · 최근 10분 0.06
- ② 어디서 보나 핵심 업무 흐름 대시보드 링크
- ③ 먼저 볼 것 Trace 쿼리 → 로그의 trace_id

알림 경로

모니터 22개



SNS 토픽



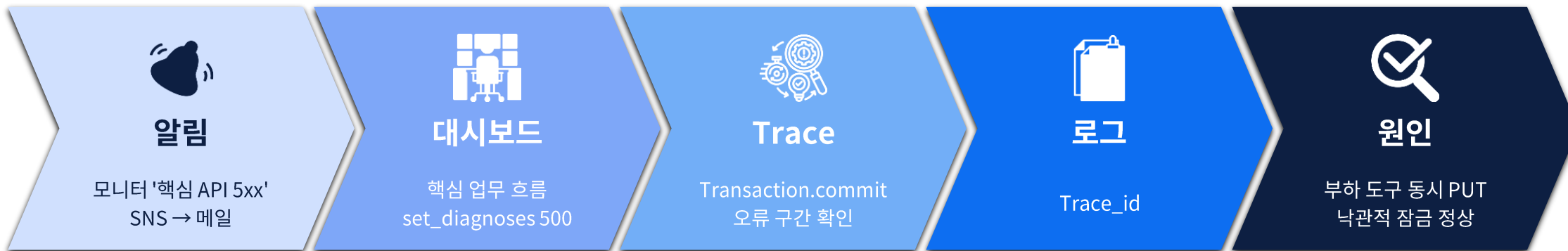
메일

증상 먼저

잠깐 튀는 건 제외

알림 = 조사 시작점

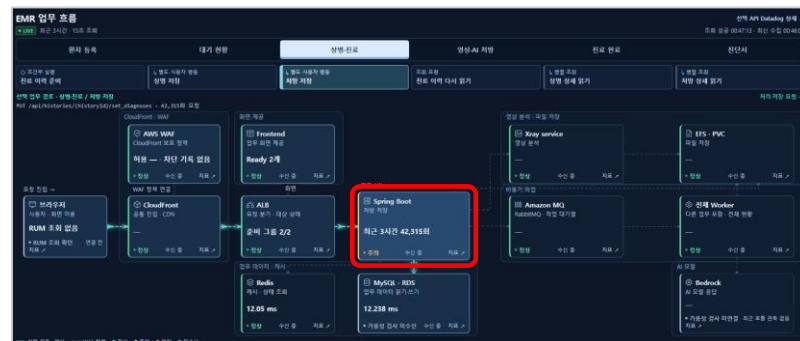
알림에서 원인 추적까지



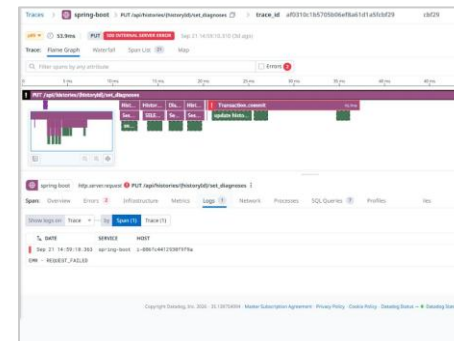
① 알림 메일 · 9/21 15:00



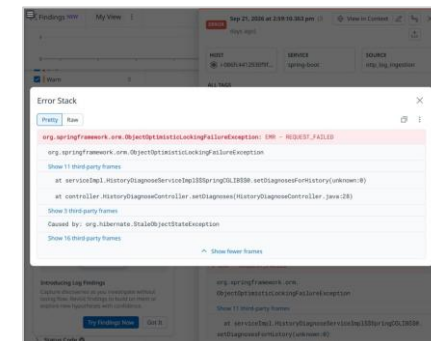
② 핵심 업무 흐름 · 처방 저장 '주의'



③ Trace · commit 실패

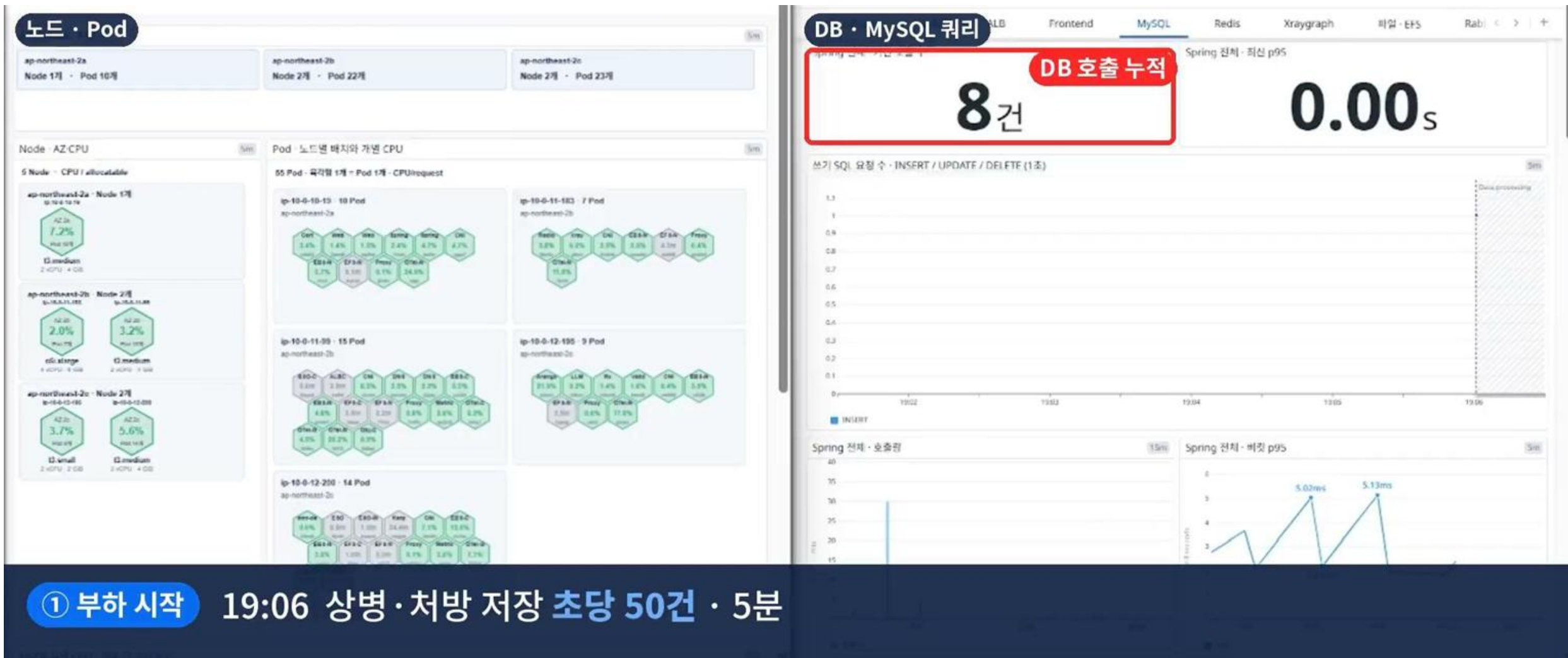


④ 로그 · 예외 · 호출 코드



원인은 부하 도구의 동시 수정: **충돌 시 변경 내용 확인 안내**

쓰기 부하에서 확장과 병목을 구분하는 과정

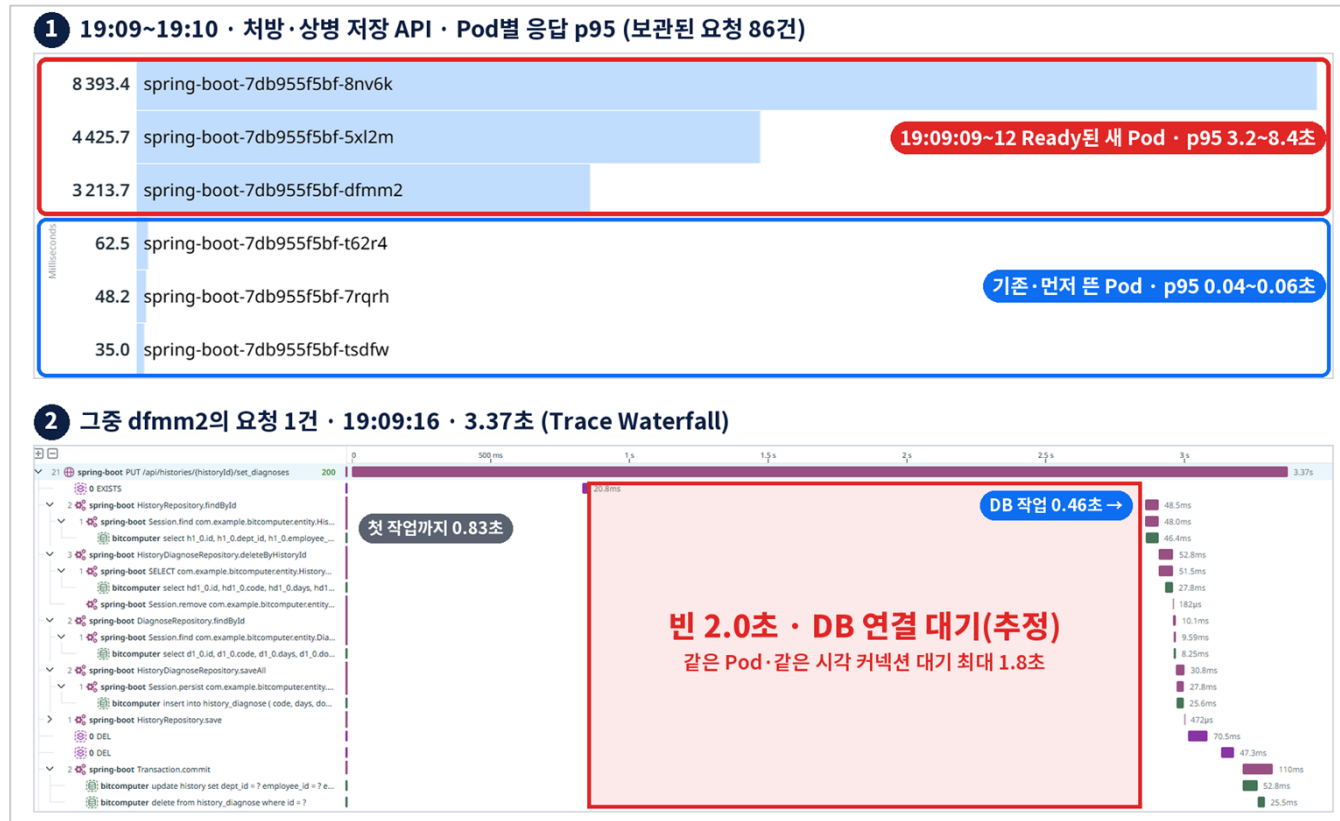


① 부하 시작 19:06 상병 · 처방 저장 초당 50건 · 5분

성공 응답에서도 느린 요청을 조사

HTTP 200만으로 정상 속도를 보장하지 않습니다.

Pod별로 나누고, 느린 요청 하나의 처리 구간을 열었습니다.



2026-09-24 19:09:16 · 3.37초

확인된 정보

같은 1분 · 새 Pod 3개만 p95 3.2~8.4초

첫 DB 호출 전 빈 2.0초

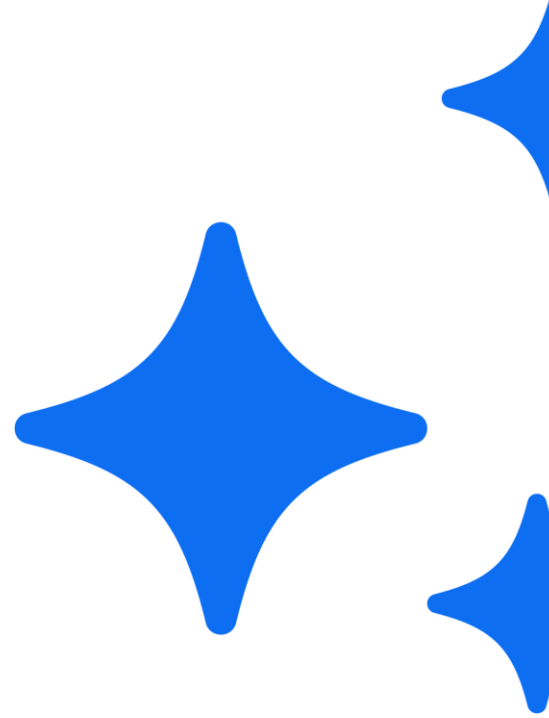
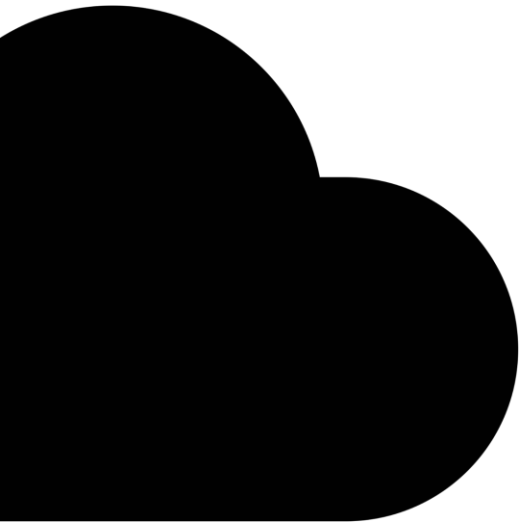
→ DB 연결 대기

DB 작업 자체는 0.46초

BESPIN GLOBAL

DR · GCP Pilot Light

홍가흔



DR 범위 : 진료 기록 읽기·쓰기

병원이 멈추면 안 되는 일만 살리고, AI 기능은 잠시 멈춥니다.

구분	✓ 살림 · GCP 에서 다시 띄움	× 멈춤 · DR 에서 제외
업무	환자 접수 · 진료 기록 조회·저장 · 상병·처방 코드 검색	AI 처방 추천·검증 · 진단서 초안 · X-ray 유사 사례
구성	frontend · spring-boot · Cloud SQL · Redis(새로 만듦) · Filestore(파일 복사 + 체크섬)	AI 서비스 6개 · RabbitMQ · ArangoDB

끄는 방법



DR 전략 선택 : Pilot Light

목표 **RTO 2시간 · RPO 1분**을 지키면서 평소 비용이 가장 낮은 방식입니다.

Pilot Light 데이터 복제와 연결만 평소에 켜 두고, 서버(GKE·앱)는 AWS 장애 때 만듭니다

전략	평소 켜 두는 것	장애 때 할 일	잃는 데이터	평소 비용	우리 조건
백업·복원	백업 파일	인프라 생성 + 백업 복원	마지막 백업 이후 전부	가장 낮음	✗ RPO 1분 불가
Pilot Light	VPN · Cloud SQL 복제본(연속 복제) 이미지 · Terraform state	GKE·앱 생성 → DB 승격 → DNS 전환	복제 지연만큼	낮음	✓ 선택
Warm Standby	위 + 작은 GKE·앱 상시 운영	규모 확대 → DNS 전환	복제 지연만큼	높음	✗ GKE·Redis·Filestore·LB 상시 비용
Active-Active	양쪽 모두 서비스	거의 없음	거의 0	가장 높음	✗ 양쪽 동시 쓰기, 단일 writer 원칙과 충돌

RPO 1분
→ 백업·복원 탈락



평소 비용
→ Warm Standby 탈락



한쪽만 쓴다 (단일 writer)
→ Active-Active 탈락

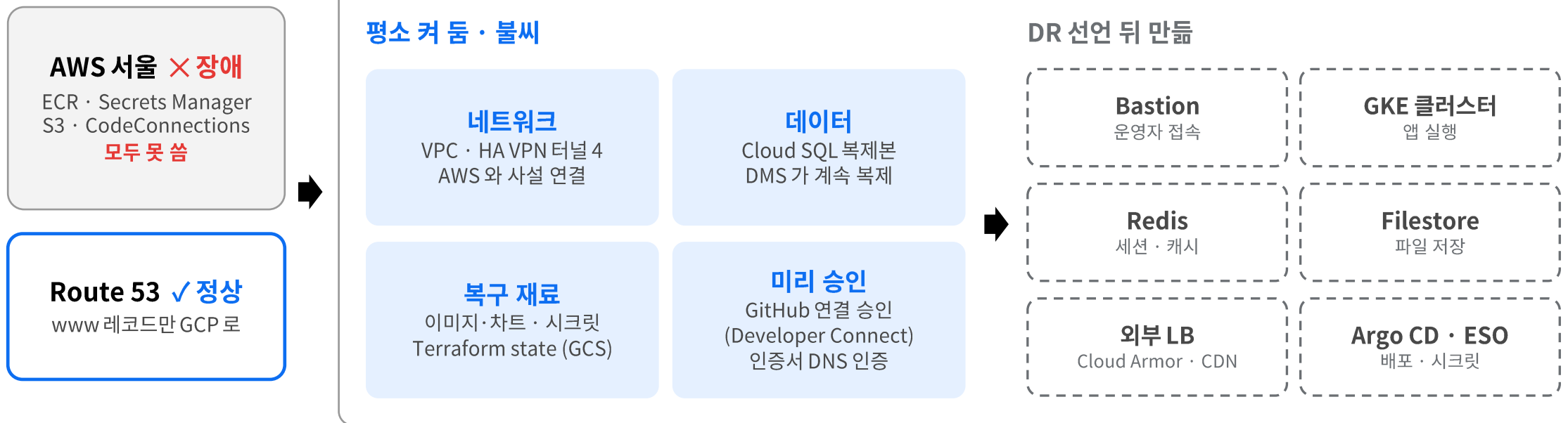


남는 것
Pilot Light ✓

Pilot Light: 평소 시 켜져있는 리소스와 DR시 생성되는 리소스

AWS 가 멈추면 AWS 의 복구 도구도 멈추므로, **복구 재료는 GCP 에 미리** 둡니다.

시나리오



DR 선언 조건

사람이 판단

① 외부 검사 실패
Synthetics 1분마다 · 밖에서 확인

② 여러 서비스 동시 영향
한 서비스 장애는 DR 아님

③ AWS 복구 예상 > 2시간
목표 RTO 보다 길 때

④ 마지막 복제 상태
지연 · 허용 손실을 기록

사람이 선언

FailOver - Pilot Light

GCP Failover 실제 전환 · 2026-09-23

Pilot Light 에서 GKE 가 뜨고 Route 53 이 넘어가기까지

1 인프라 · GKE 기동



2 권한 · GitOps 연결



3 DB 승격 · 앱 활성화



4 검증 · Route 53 전환

사람이 입력하는 결정 두 번 · **PROMOTE** DB 승격 · **CUTOVER** DNS 전환

녹화 4개 · 대기 구간 생략 · 시각은 화면 출력과 GitHub 기록 기준(KST)

RTO·RPO : 목표와 9/23 훈련 실측

9/23 훈련에서 밖에서 본 중단은 **70분**, RPO는 **1분** 미만으로 목표 부합

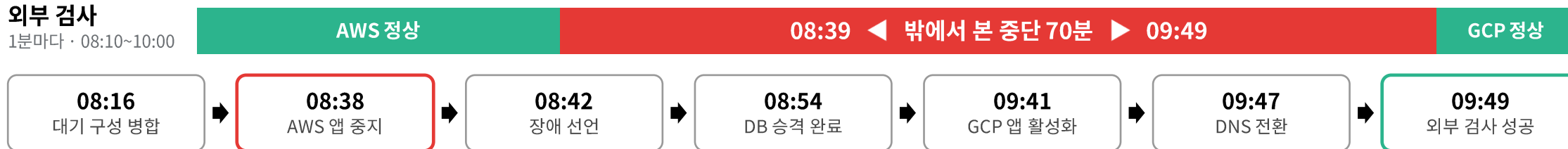
RTO 장애 시작 → 밖에서 서비스 응답이 돌아올 때까지

RPO 복구할 때 잃을 수 있는 데이터의 시간 폭

구분	RTO · 복구 시간	RPO · 잃는 데이터 폭
목표 (팀 제안)	2시간	1분
9/23 훈련 실측	70분 외부 검사 08:39 첫 실패 → 09:49 성공	1분 미만 쓰기 차단 뒤 복제 지연 0초 3회 확인 → 승격
평상시 측정	—	복제 지연 p50 0.14초 · p95 0.22초 (표본 30개)

외부 검사

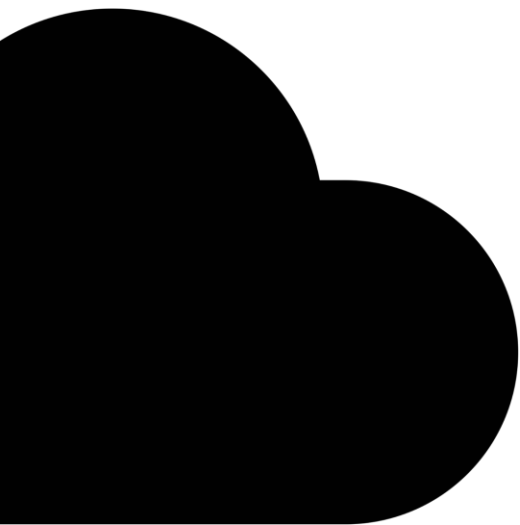
1분마다 · 08:10~10:00



BESPIN GLOBAL

OPSNOW

홍가흔



OpsNow 비용 관리 : AWS · GCP

OpsNow

1

예상 비용 · 추이 분석

지난 사용 패턴으로 이번 달 비용을
예측해 예산 초과를 막습니다

2

이상 비용 탐지

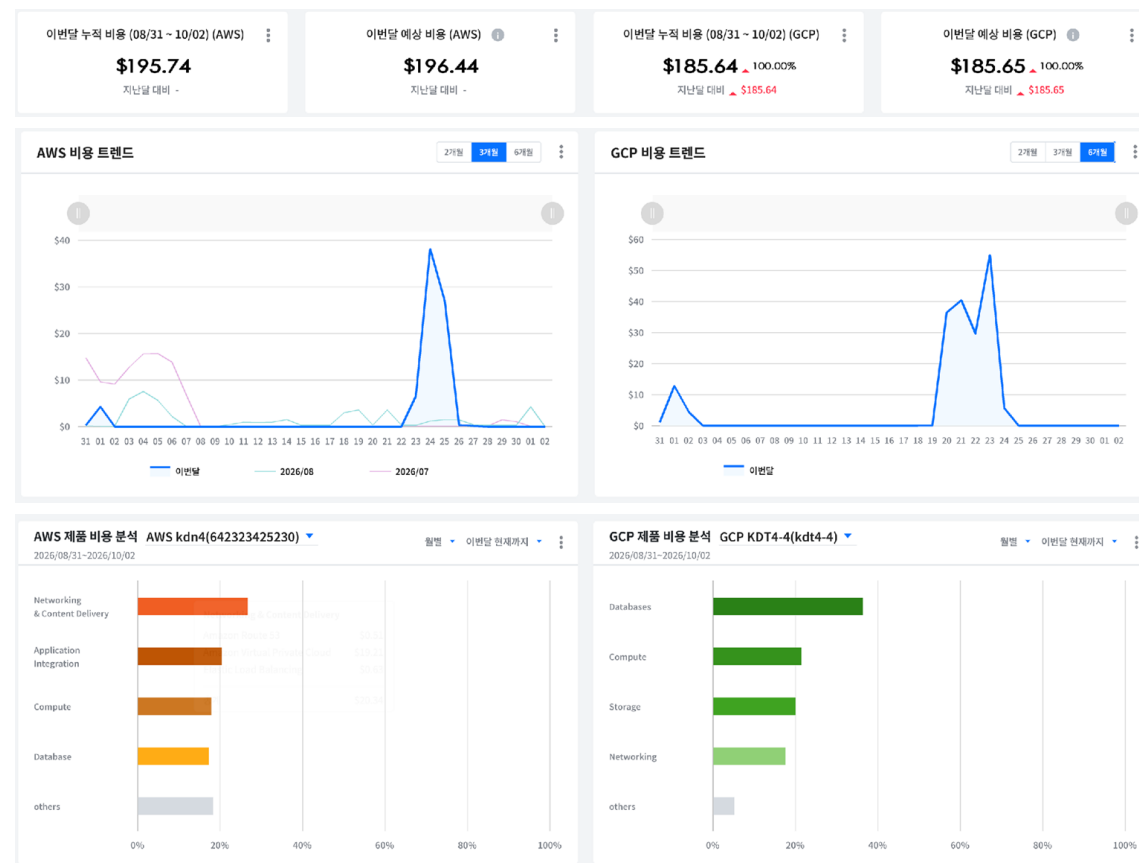
갑작스러운 자원 남용이나
청구액 급증을 바로 잡아냅니다

3

통합 대시보드

AWS · GCP에 흩어진 비용을
한 화면에 모읍니다

프로젝트 실제 비용 (8/31~10/2)



원칙은 끝까지 지켜졌고, 대가와 결과는 숫자로 남았습니다

NAT · 인터넷 Egress 없음은 제약이 아니라 설계 기준

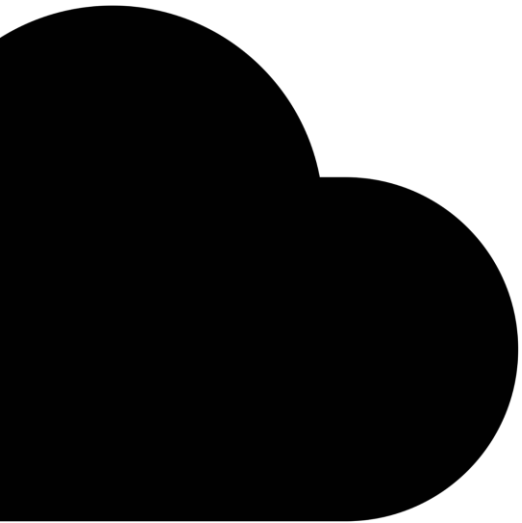


“ NAT 없이, 환자 기록을 다루는 서비스를 늘리고 · 바꾸고 · 살렸습니다 ”

BESPIN GLOBAL

Thank You

HELPING YOU ADOPT CLOUD & AI



팀원별 역할 & 회고

홍 가 훈 SRE

M 010-4081-4408
E honggaheun@gmail.com
서울특별시 서초구 서초대로74길 33



Observability · DR · Cost Analysis

최종 프로젝트를 NAT없이 진행하면서 Private 통신으로만 서비스가 가능하게 하였습니다. 그 과정에서 데이터의 흐름이 어떻게 되는지 파악하면서 클라우드와 인프라 전반에 대한 이해도를 높일 수 있는 시간이었습니다.

김 동 훈 DevOps Engineer

M 010-7400-5708
E kjbdo@gmail.com
서울특별시 서초구 서초대로74길 33



Kubernetes · CI/CD · GitOps

쿠버네티스를 직접 운영해 보거나 CI/CD 파이프라인을 구축해보고, NAT 없는 환경에서 아웃바운드에 기대던 서비스를 하나씩 찾아내며, 교육과정 전반에 걸쳐 개발만큼 인프라를 이해하는 것 IT 운용에 있어 중요하다는 것을 배웠습니다.

강 지 현 Cloud Infrastructure Engineer

M 010-2627-6570
E qounggcur@gmail.com
서울특별시 서초구 서초대로74길 33



Terraform · Iac

프로젝트를 진행하며 인프라는 필요한 자원을 구축하는 데서 끝나는 것이 아니라, 안정적인 시스템을 위해 배포·운영 요소 간의 연결도 함께 살펴야 한다는 것을 배웠습니다. 전체 흐름을 이해하고 구성하는 경험을 통해 인프라를 더 넓은 관점에서 바라볼 수 있었습니다.

이 성 윤 Cloud Architect

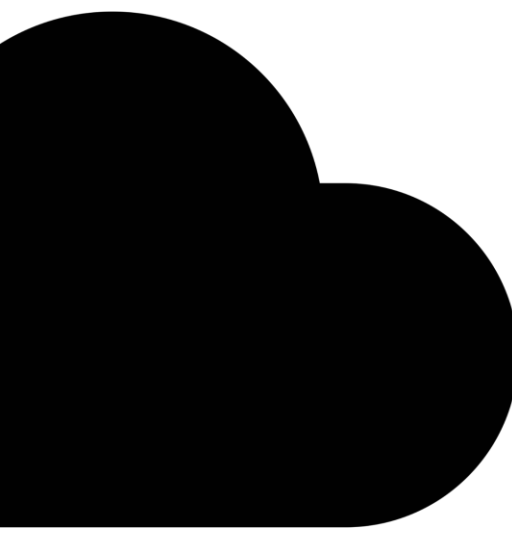
M 010-5402-4200
E lucia25834@gmail.com
서울특별시 서초구 서초대로74길 33



Cloud Architect · System Design

프로젝트 과정에서 인프라에 실제로 필요한 네트워크, 권한, 배포 설정 과정에 참여해 보며 클라우드 인프라 전반을 폭넓게 이해할 수 있었습니다. 제약에 맞추어 대안을 찾아 설계하는 것 또한 중요함을 배웠습니다.

BESPIN GLOBAL



#1. 부록 EKS 버전 업그레이드

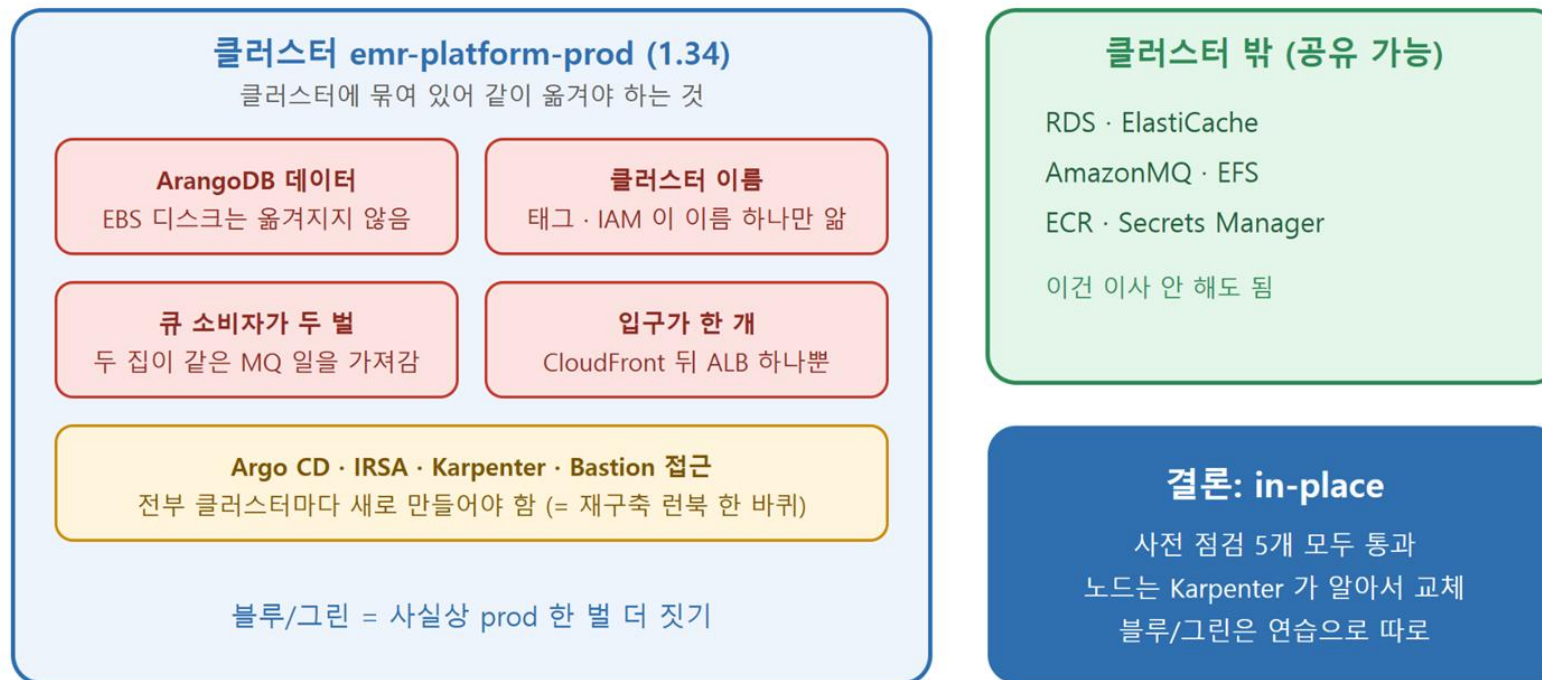
김동훈



In-place instead of blue/green

1.34 표준 지원이 **2026-12-02** 에 끝납니다. 두 방법을 비교하고 in-place 를 골랐습니다.

우리 인프라에서 새 집으로 이사가 어려운 이유



1.34 표준 지원 종료 2026-12-02 · 그 뒤엔 연장 지원 요금

준비 : 업그레이드 당일 drift = “0”

PDB · preStop · AMI 고정은 **평소에 이미 커 둔 설정**입니다. 당일엔 잠그고, 버전표를 대조했습니다.

평시 장치 확인



무중단 설정

PDB · preStop 15초 · HPA min 2
AMI 날짜 고정 · drift 한 대씩
업그레이드용이 아니라 평소 설정

drift 잠금



Drifted nodes: "0"

AMI 별칭은 클러스터 버전으로 풀려서, 컨트롤 플레인을 올리는 순간 모든 Karpenter 노드가 drift 됩니다. kubectl로 고치면 Argo가 되돌려 git PR로 잠금

호환 확인



올리기 전에 버전표부터

Karpenter 1.14.1 (1.35는 1.9 이상)
LBC 3.5.0 · OTel 차트 통과
ESO 2.10은 공식 테스트 1.36 뿐, 실제 동작
1.35에서 제거되는 API 없음

- ❑ 끊기면 안 되는 것 : frontend · spring-boot (각 2개 + PDB)
- ❑ 끊겨도 되는 것 : AI 6개 · ArangoDB (노드가 바뀔 때 잠깐 멈춤)

순서 : 컨트롤 플레인, 애드온, 노드.

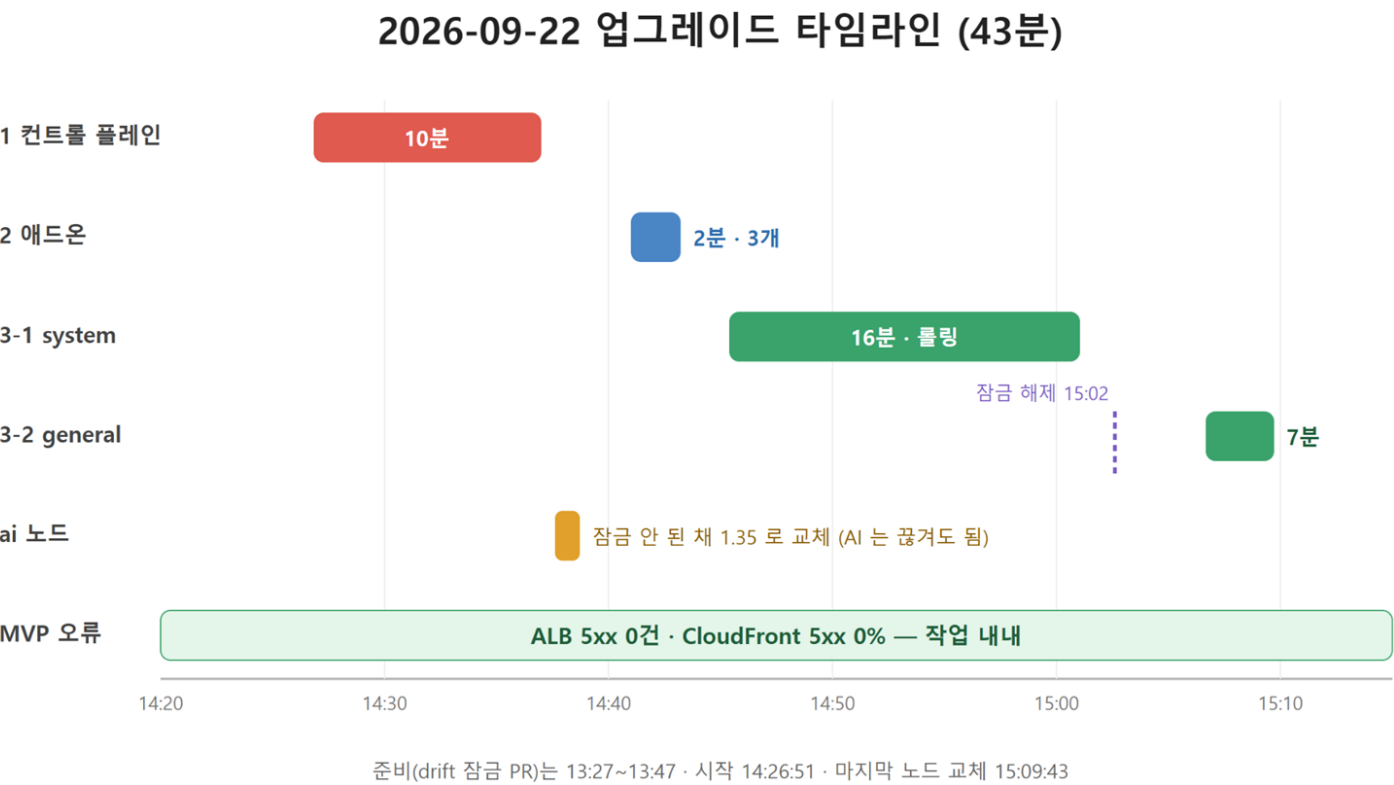
콘솔과 git PR 로 진행했습니다. 컨트롤 플레인 **되돌릴 수 없어** 준비가 끝난 뒤에만 눌렀습니다.



- 1** 순서가 곧 규칙 : kubelet 은 API 서버보다 새 버전일 수 없어 컨트롤 플레인이 항상 먼저입니다
- 2** 애드온은 하나씩 : ebs · efs CSI · metrics-server 는 이미 1.35 호환 버전이라 그대로 뒀습니다
- 3** 런북과 달랐던 점 : ai NodePool 을 잠그지 않아 컨트롤 플레인 직후 ai 노드가 먼저 교체됐습니다(AI 는 다운타임 감수 대상)

EKS 1.34 → 1.35 결과

컨트롤 플레인 10분 · 애드온 2분 · system 노드 16분 · Karpenter 노드 7분. **ALB 5xx 0** · **CloudFront 5xx 0%**



ALB 5xx 0건

CloudFront 5xx 0%

노드 5대 모두 1.35

Argo CD 앱 14 Synced · Healthy

※ 컨트롤 플레인 1.35.8-eks.24 · Drifted 노드 0 · 옛 계정(987622176301) 2026-09-22 작업

시연 : 콘솔에서 버전을 올리고 노드가 한 대씩 바뀌는 모습

```
30     group: karpenter.k8s.aws
31     kind: EC2NodeClass
32     name: default
33     expireAfter: 720h
34     # **중요! 상한이다, 노드 개수가 아니다.** cpu: "8" 은 이 NodePool 전체 노드의
35     # vCPU 합이 8을 넘지 않게 막는다는 뜻이다 - t3.medium(2 vCPU) 이라면 최대 4대,
36     # t3.small(2 vCPU) 이라면 마찬가지로 4대다. 코드 스니펫에 이 상한을 넣기엔
37     # Karpenter 는 새 노드를 만들지 않고 Pending 으로 본다.
38     limits:
39       cpu: "8"
40       memory: 16Gi
41     disruption:
42       # 빈 노드는 즉시, 덜 찬 노드는 재배치해 통합한다(가용량이지만 명시한다).
43       consolidationPolicy: WhenEmptyOrUnderutilized
44       consolidateAfter: 1m
45       # **drift(AMI 교체)는 한 번에 한 대씩.** EC2NodeClass 의 AMI 날짜를 올리거나
46       # 컨트롤 플레인 버전을 올릴 때마다 노드 전체가 drift 된다.
47       # frontend-spring-boot 의 PDB 와 함께 두 리플리카가 차례로 옮겨지게 한다.
48       #
49       # budgets 를 먹으면 가용량(nodes 10%)이 사라진다. reasons 라는 것 없이 그
50       # 가용량을 그대로 되살린다 - 없으면 empty-underutilized 에 예산이 들어간다.
51       #
52       # expireAfter(720h) 안으로 강제 교체라 이 예산을 먹은지 막는다.
53       # 클러스터 업그레이드 용임에만 drifted 를 "8" 으로 줄인다(노선 합계 1.34 + 1.35).
54     budgets:
55       - nodeclass="t3"
56       - nodes: "1"
57       reasons: ["Drifted"]
58
```

Use **Control + Shift + ⌘** to toggle the **tab** key moving focus. Alternatively, use **esc** then **tab** to move to the next interactive element on the page.

1

준비 · drift 잠금

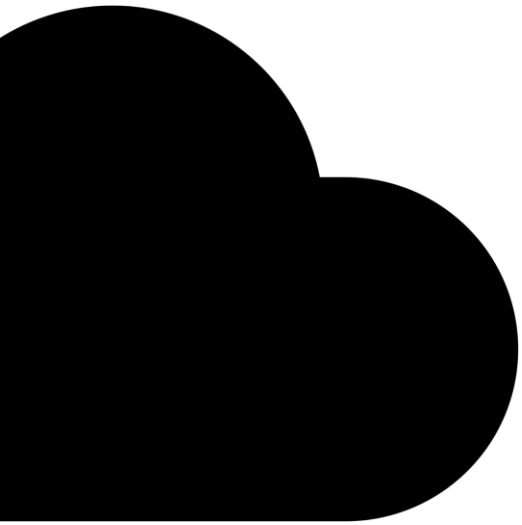
컨트롤 플레인을 올리는 동안 Karpenter 가 노드를 자동 교체하지 않게 교체 예산을 '0'으로 잠급니다

BitComputer EMR · www.bespin-global.cloud

1 / 8 ● ○ ○ ○ ○ ○ ○ ○

BESPIN GLOBAL

#2. 부록 상세 기술 설명



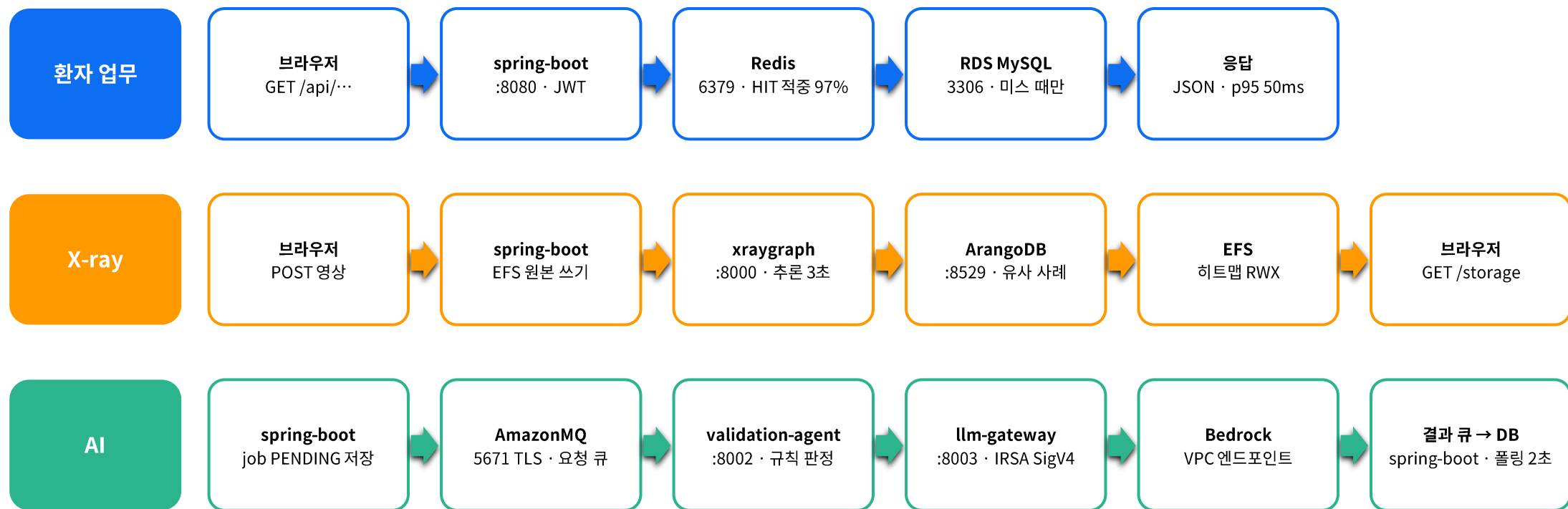
경로 생성 : Terraform, LBC 와 GitOps를 통해 만듭니다

트래픽이 지나는 구간마다 **만든 주체와 코드 위치**가 다릅니다: II 장은 여기서 시작합니다.

구간	프로토콜 · 포트 · 규칙	만든 주체	코드 위치
브라우저 → CloudFront	HTTPS 443 · TLS 1.2_2021 · WAF 관리형 규칙 2	Terraform (edge)	infra 40-platform · 20-security
CloudFront → 내부 ALB	VPC Origin · HTTP 80 · ALB ARN 입력	Terraform (edge)	40-platform/20-cloudfront.tf
ALB → 파드	HTTPRoute 3 · targetType ip · 헬스체크	LBC (Gateway API)	gitops charts/.../gateway.yaml
파드 ↔ 파드	ClusterIP HTTP · 이름 = compose 컨테이너 이름	GitOps 차트	gitops values.yaml 서비스 간 주소
파드 → RDS · Redis · MQ · EFS	3306 · 6379 · 5671 TLS · 2049 · SG 는 app 서브넷 CIDR 만	Terraform (foundation)	20-security · 30-data
파드 → AWS API	Interface 엔드포인트 14 + S3 GW · IRSA 토큰	Terraform + eksctl	15-vpc-endpoints · eksctl IRSA 5
클러스터 → Datadog	PrivateLink 4 · 인터넷 0	Terraform	15-vpc-endpoints (AZ ID 매핑)
운영자 → 클러스터	SSM 세션 · 인바운드 0 · kubectl 은 Bastion 에서만	Terraform	40-platform Bastion · 20-security IAM

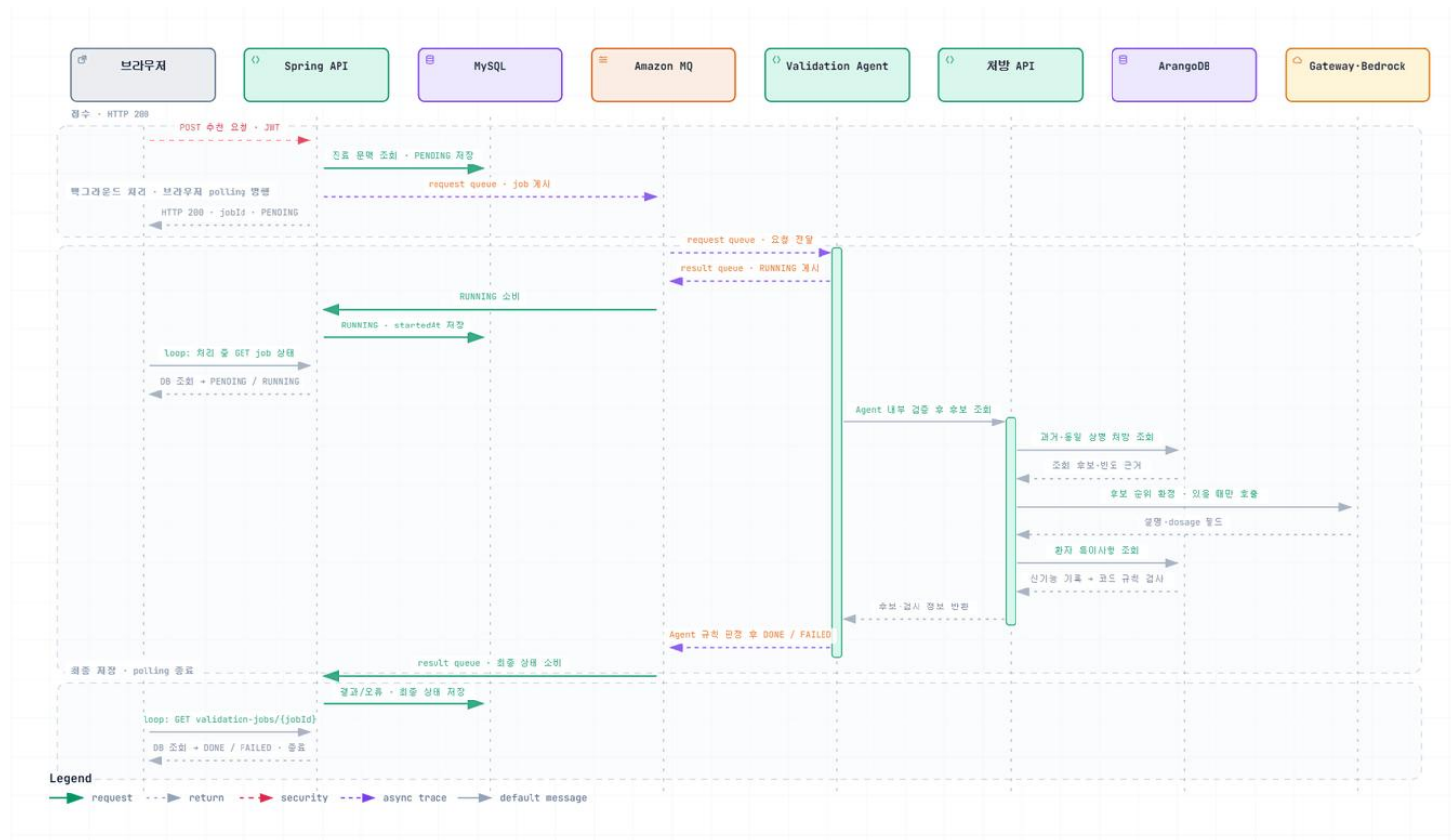
클러스터 안 3가지 작업 : 업무 / X-ray / AI 작업 (큐, 비동기)

처방 검증·진단서는 **202 + jobId** 를 돌려주고 큐로 넘어갑니다. 결과는 spring-boot 가 DB 에 씁니다.



큐에 넣고 바로 응답합니다 : 상태는 DB 에 있고 파드에는 없습니다

검증 요청은 **job 행(PENDING)**과 **큐 메시지**로 남고, 브라우저는 jobId 로 DB 를 폴링합니다. 그래서 어느 파드가 죽거나 늘어나도 작업이 이어집니다.



1 상태는 DB 한 곳

job 행이 PENDING → RUNNING → DONE / FAILED. 파드 메모리에는 상태가 없어 폴링은 어느 spring-boot 가 받아도 같은 답

2 요청은 durable 큐

처리를 마친 뒤에 ack. 작업 중 파드가 죽으면 메시지가 다른 validation-agent로 재전달 (prefetch 1)

3 진행도 큐로

agent는 RUNNING · DONE / FAILED 를 결과 큐에 발행만 하고, DB 에 쓰는 건 spring-boot. agent는 DB 를 모름

4 같은 jobId 가 동시에 와도

결과 소비자가 행을 잠그고 바꿔(PESSIMISTIC_WRITE), 늦게 온 RUNNING 이 먼저 끝난 DONE 을 덮지 않음

준비 상태는 readiness(), 포함 여부는 count로 제어

선택한 단계에 필요한 AWS 자원의 준비 상태는 readiness()로 확인하고, 단계별 설정값에 따라 count로 조회와 자원을 포함합니다.

```
# modules/45-karpenter/10-data.tf
data "aws_eks_cluster" "this" {
  count = var.enable_karpenter ? 1 : 0 # 클러스터가 있어야 읽는다
  name = var.cluster_name
}
data "aws_iam_openid_connect_provider" "this" {
  count = var.enable_karpenter ? 1 : 0 # OIDC 도 eksctl 이 만든다
}

# modules/47-argocd/30-cluster-access.tf
resource "aws_eks_access_policy_association" "argocd_capability" {
  count = var.enable_cluster_access ? 1 : 0 # access entry 는 capability 가
}

# modules/40-platform/20-cloudfront.tf
data "aws_lb" "this" { count = var.enable_cloudfront ? 1 : 0 }
resource "aws_cloudfront_vpc_origin" "alb" { count = var.enable_cloudfront ? 1 : 0 }
resource "aws_cloudfront_distribution" "this" { count = var.enable_cloudfront ? 1 : 0 }
```

단계별 포함 항목	readiness()가 확인할 조건
Karpenter · Bastion 접근	EKS가 실행 중이고 OIDC가 준비
Argo CD 접근	Argo CD 기능이 실행 중
CloudFront + ALB 주소	내부 ALB 생성 여부

- ❑ 단계별 설정값이 count를 결정합니다.
- ❑ count = 0이면 해당 조회·자원을 제외하고, 1이면 포함합니다.

※ readiness()는 자체 제작한 스크립트의 함수로, AWS 자원의 존재와 준비 상태를 불러오는 역할을 합니다.

handoff : 결과값을 다음 도구에 넘기기

Terraform 출력 중 검증한 비밀 아닌 값만 골라 다음 도구의 설정 파일로 만듭니다.

파일	내용	받는 곳
outputs.json	검증한 Terraform 결과값	검토자
provenance.json	어떤 plan · 코드에서 나왔는지	감사 기록
cluster-prod.yaml	VPC · 서브넷 · 권한 정보	eksctl create cluster
values-infra.yaml	서비스 주소 · 스토리지 · 권한 정보	GitOps envs/prod PR
argocd-capability.yaml	Argo CD 기능 설정	eksctl create capability
requirements.json	사람이 마저 할 일	운영자

※ 전체 raw output은 올리지 않습니다. 단계에 맞는 파일만 검토해 전달합니다.

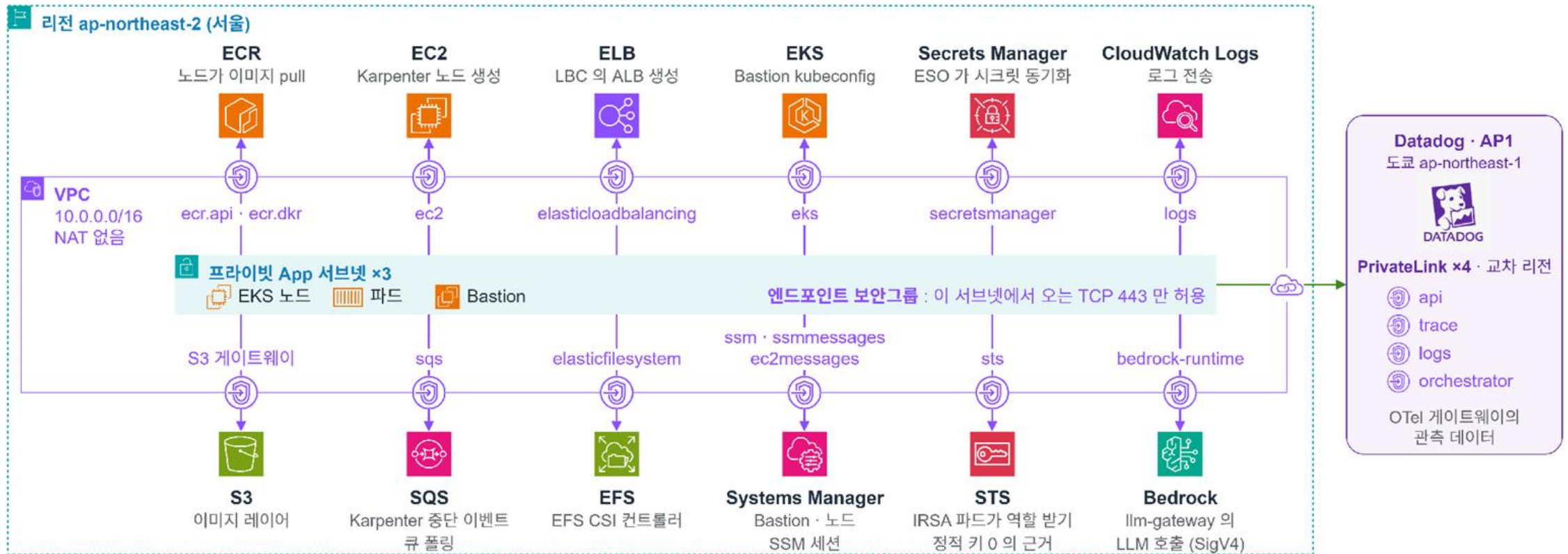
사람이 이어서 하는 일

- ① EKS · Argo CD 기능 생성
- ② GitHub 연결 승인
- ③ 비밀값 · GitOps PR 검토

III Kubernetes

나가는 길 : NAT 대신 엔드포인트

엔드포인트 보안그룹은 **App 서브넷의 443 만** 받습니다.



Argo CD 가 sync-wave 순서대로 클러스터를 채웁니다

Application의 **의존 관계**대로 애플리케이션을 배포합니다.

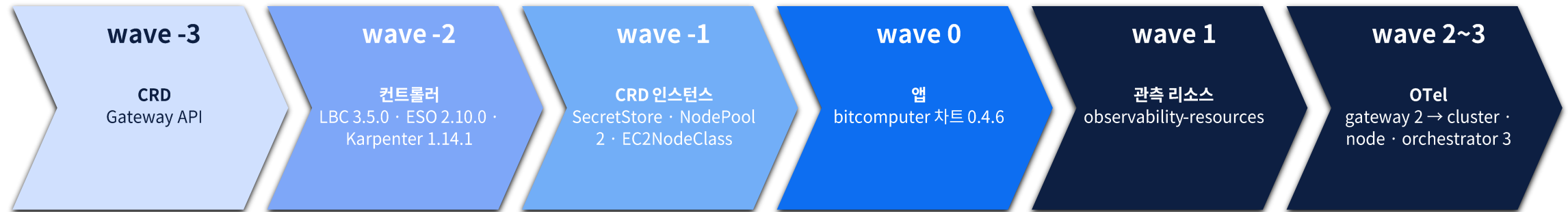
※ bootstrap 매니페스트 3개

- cluster-in-cluster.yaml
- appproject-default.yaml
- root-app.yaml

: Argo CD에 배포 대상 클러스터를 등록한다. 관리형이라 주소가 URL이 아니라 클러스터 ARN이다

: 어느 저장소에서 읽어 어느 클러스터·네임스페이스에 배포해도 되는지 허용 범위를 정한다

: gitops 레포의 argocd/ 폴더를 읽으라고 지시한다. 여기서 나머지 Application 13개가 줄줄이 생긴다 (app-of-apps)



- 1** 없는 종류는 만들 수 없다: CRD(-3) 가 먼저 있어야 API 서버가 Gateway · NodePool 객체를 받아 줍니다
- 2** 읽을 주인이 먼저: 컨트롤러(-2) 가 떠 있어야 NodePool(-1) 이 노드로, Gateway 가 ALB 로 바뀝니다
- 3** 앱은 마지막: wave 0 파드가 Pending 이 되면 그때 Karpenter 가 노드를 띄웁니다. 데이터 적재 Job 은 차트 안 wave 5

※ 관리형 Argo CD 는 sync 제한이 120초라 적재 Job 은 혹 대신 일반 Job + 재시도 10회. Git 은 CodeConnections 로 읽기만, 클러스터 안에 GitHub 토큰 없음

노드가 바뀌어도 frontend · spring-boot 는 끊기지 않게 묶었습니다

drift · 통합 · 만료 · 업그레이드, **노드가 바뀌는 모든 경로**에서 한 번에 한 개만 내려가게 했습니다.

장치	설정	막는 것	막지 못하는 것
AMI 날짜 고정	al2023@v20260917	@latest 가 몰래 전 노드를 새 AMI 로 교체	패치는 날짜를 올리는 PR 로
drift 예산	Drifted 1대 · 그 외 10%	여러 노드 동시 교체	—
30일 만료	expireAfter 720h	오래된 노드 방치	예산과 무관하게 시작 : 속도는 PDB 가 제어
PDB	minAvailable 1 (두 앱)	drain · 업그레이드 때 둘 다 내려감	노드 장애 같은 비자발 중단
preStop	sleep 15초	ALB 대상 해제 전에 파드가 죽어 요청 유실	—
최소 레플리카	HPA min 2	재기동 순간 503 (09-07 겪음)	—

- ❑ **한계** : 파드를 노드 · 존에 흩뜨리는 설정(topology spread)이 없어 두 레플리카가 한 노드에 같이 뜰 수 있습니다
- ❑ **범위** : AI 6개 · ArangoDB 는 다운타임을 감수하는 대상입니다

중단 이벤트 : EventBridge 가 큐에 넣고, Karpenter 가 꺼내 갑니다

EC2 쪽에서 노드가 사라지기 전에 **파드를 먼저 옮기기** 위해서입니다. NAT 없이 SQS 엔드포인트로 폴링합니다.



EventBridge 규칙	무엇을 알리나	우리 NodePool 에서
AWS Health Event	예정 정비 · 인스턴스 리타이어	온다 : 정비 전에 미리 비움
EC2 Instance State-change	Karpenter 밖에서 일어난 중지 · 종료	온다 : 사라진 노드 정리
EC2 Spot Interruption Warning	Spot 회수 2분 전 예고	오지 않음 : on-demand 전용
EC2 Rebalance Recommendation	Spot 회수 위험 증가	오지 않음 : on-demand 전용
Capacity Reservation Interruption	용량 예약 회수	오지 않음 : 예약 미사용

※ 큐 · 규칙 · 권한은 Terraform 45-karpenter(cluster 단계). 보존이 300초라 컨트롤러가 5분 넘게 멈추면 그 사이 이벤트는 사라집니다

부하에서 Ready 까지 : 단계 사이마다 설정 하나가 움직입니다

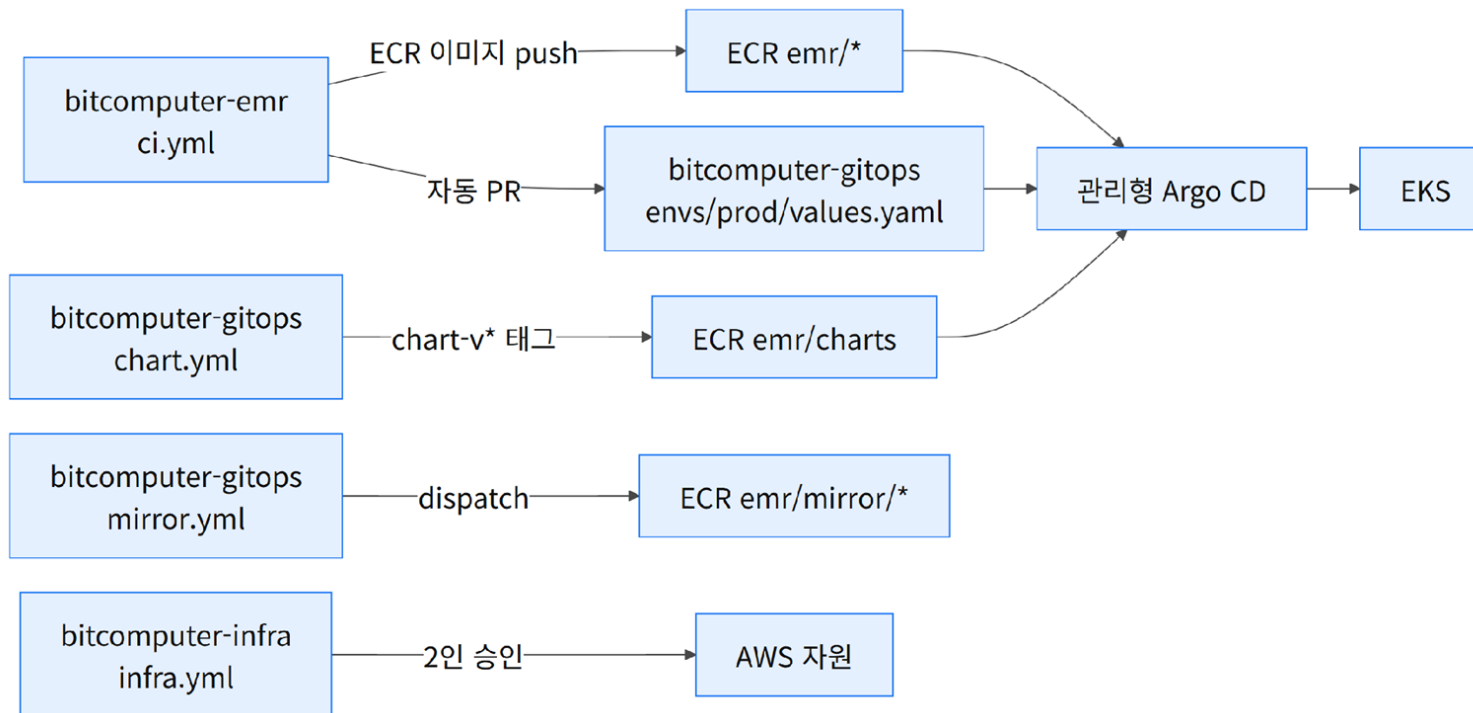
부하테스트 예시 : 파드 2 → 6 · 노드 5 → 7 · desired 6 에서 available 6 까지 약 2분



줄어들 때: HPA 5분 안정화 뒤 축소 → 빈 노드는 통합 1분 뒤 정리 → PDB 가 한 번에 한 개만 내리게 함

세 레포의 산출물이 Argo CD 한 곳으로 모입니다

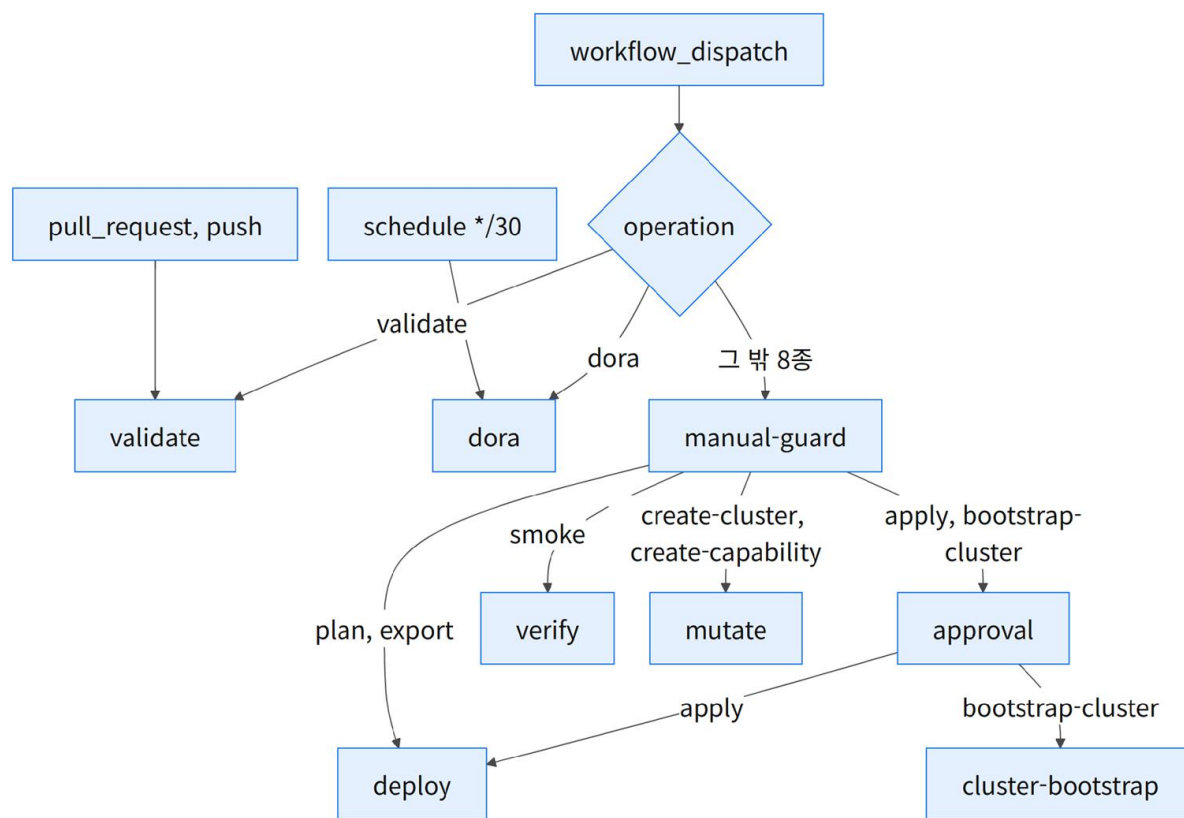
레포끼리는 서로 부르지 않고 **ECR과 git에 놓인 산출물로만** 이어집니다. 이 경계가 곧 자격증명 경계입니다.



“ 앱은 이미지를, gitops 는 차트와 값을, infra 는 AWS 자원을 : **각자 자기 것만** 만듭니다 ”

infra ① PR 은 검증만, 실제 운영은 dispatch 로 고른 operation 하나

operation 마다 지나는 잡이 다릅니다. 자원을 바꾸는 **apply** 와 **bootstrap-cluster** 만 승인 게이트를 지납니다.

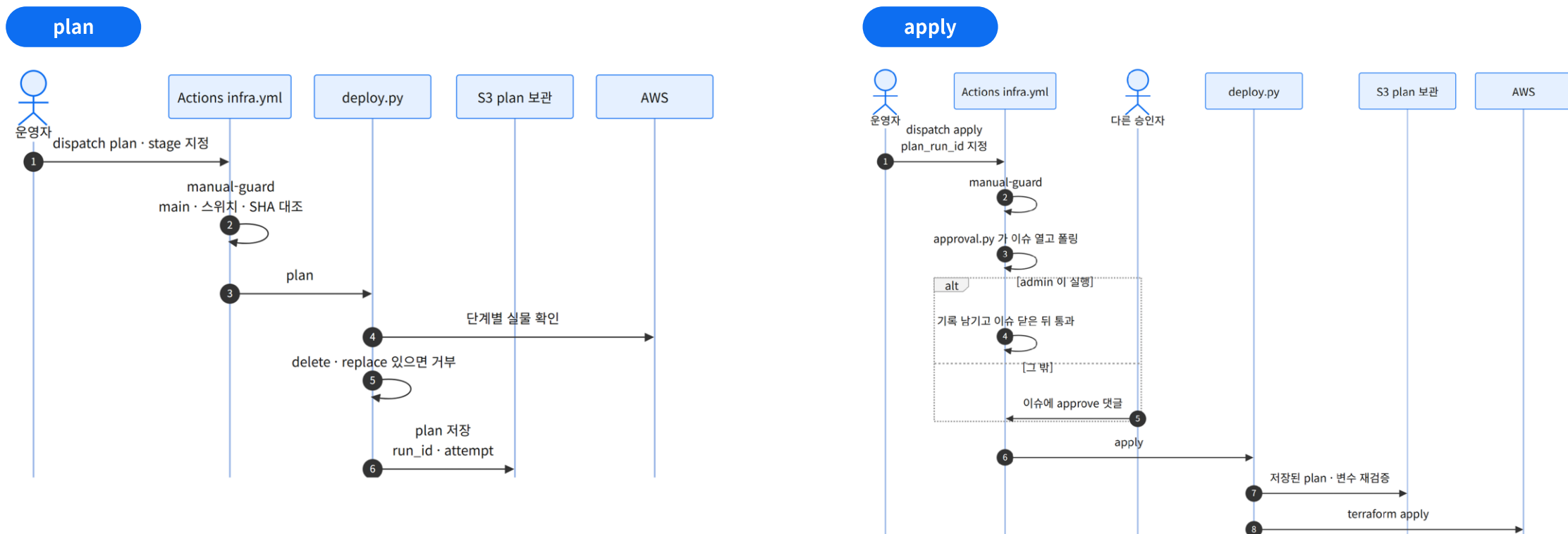


operation	하는 일	승인
validate	정적 검증 · mock plan	없음
plan · export	plan 저장 · output 만	없음
apply	저장한 plan 그대로 적용	필요
bootstrap-cluster	bootstrap 3 객체 적용	필요
smoke · dora	읽기 전용 검사 · 지표	없음
create-*	eksctl 생성	dev 만

※ PR · push 는 validate 만 돈다. 마스터 스위치 AWS_OPERATIONS_ENABLED 가 꺼지면 dispatch 도 전부 멈춘다

infra ② 저장한 plan 만 apply 되고, 그 사이에 승인 이슈가 섭니다

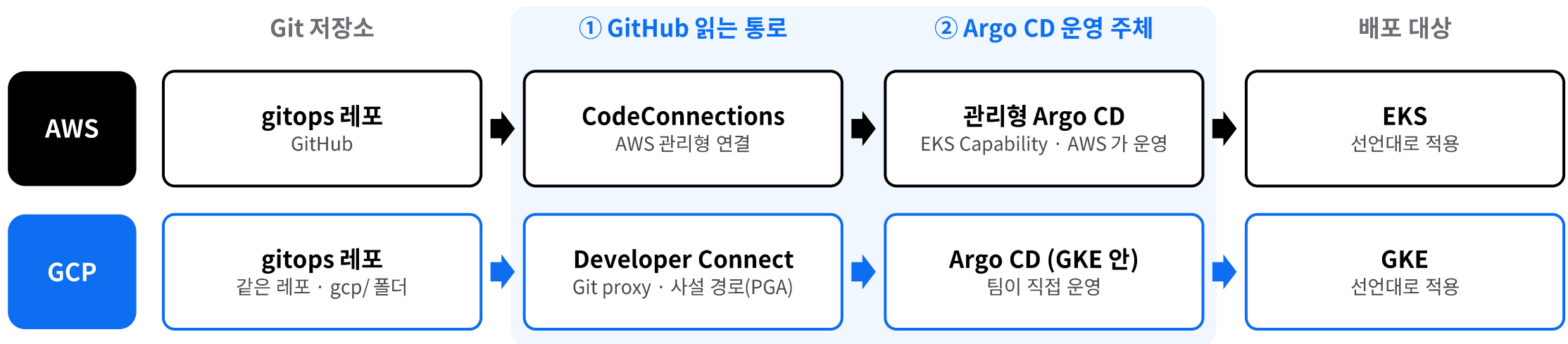
plan 과 apply 는 다른 실행입니다. 단 **저장소 admin** 은 기록만 남기고 통과합니다.



※ admin 분기는 운영 브랜치 #99(09-24). admin 은 이미 워크플로 · 승인자 · 규칙을 바꿀 수 있어 2인 승인이 막는 것이 없었다

GCP GitOps : 같은 레포, 바뀐 점

레포·차트·사람 병합은 AWS 와 같고, **GitHub** 를 읽는 **통로**와 **Argo CD 운영 주체**만 바꿨습니다.



그대로인 것

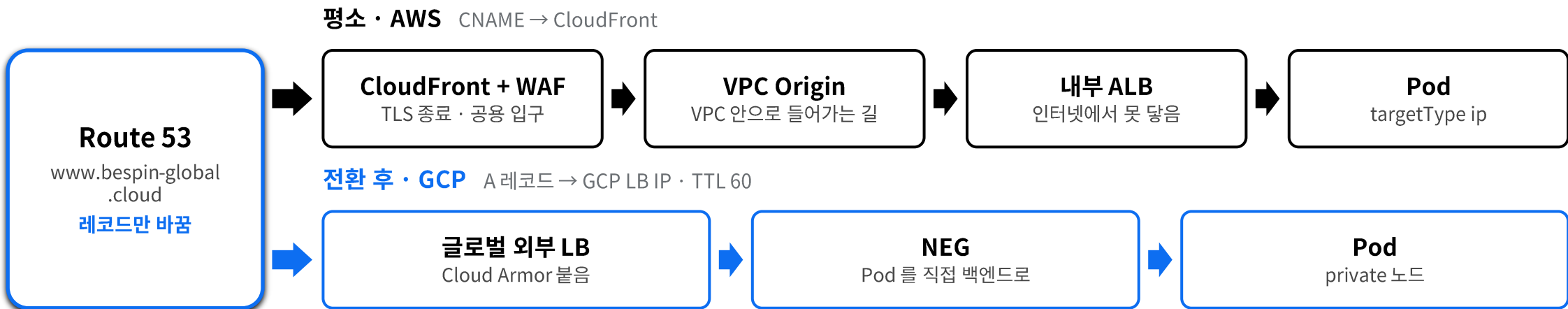
- 배포 결정
- 차트는 OCI 저장소, 환경값은 git (ECR ↔ Artifact Registry)
- NAT 없음 (VPC Endpoint ↔ Private Google Access)

GCP 에서 팀이 더 맡는 것

- Argo CD 버전 · 업그레이드
- Git·차트 읽기 토큰을 30분마다 자동 재발급 (ESO)
- Argo 화면은 공개 X → IAP 로 Bastion 경유

사용자 진입점 : 같은 주소, 레코드 하나로 전환

사용자는 같은 주소로 들어오고, 장애 때는 **Route 53 레코드 하나**만 GCP 입구로 바꿉니다.



역할	AWS · 평소	GCP · 전환 후
TLS 인증서	ACM · CloudFront 용 (us-east-1)	Certificate Manager · DNS 인증을 미리 해 둠
공격 방어	WAF 관리형 규칙 2	Cloud Armor SQLi · XSS (처음엔 preview) · IP당 분당 1000 요청 제한